

АННОТАЦИЯ

Производственная практика выполнена в форме научно-исследовательской работы на тему: «Методы обеспечения отказоустойчивости в современных информационных системах».

Объем научно-исследовательской работы – 74 страницы, на которых размещены 47 рисунков, 11 таблиц и 7 листингов. Использовано 20 источников.

Ключевые слова: отказоустойчивость, микросервисная архитектура, контейнеры, оркестраторы контейнеров, CI/CD-конвейеры, стратегии развертывания.

Объектом исследования являются микросервисные информационные приложения.

Предметом исследования является построение отказоустойчивой информационной системы с применением инструментов контейнеризации и оркестрации контейнеров.

Научно-исследовательская работа состоит из введения, трех глав и заключения.

Во введении раскрывается актуальность темы, обозначаются проблемы, цели и задачи, определяются объект и предмет исследования научно-исследовательской работы, указывается практическая значимость.

Первая глава посвящена анализу принципов и методов построения отказоустойчивых информационных систем на инфраструктурном и архитектурном уровнях и анализу лучших практик.

Вторая глава посвящена практической реализации отказоустойчивой информационной системы на микросервисной архитектуре и построению CI-конвейера, автоматизирующего процессы сборки и публикации Docker образов.

В третьей главе проводится тестирование разработанной информационной системы, в ходе которого продемонстрирована высокая

эффективность современных архитектурных решений обеспечения отказоустойчивости.

Заключение содержит выводы по работе, обобщает результаты исследования и указывает на перспективы дальнейшего развития разработанной информационной системы.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	7
ГЛАВА 1. АНАЛИЗ ПРИНЦИПОВ И МЕТОДОВ ПОСТРОЕНИЯ ОТКАЗОУСТОЙЧИВЫХ ИНФОРМАЦИОННЫХ СИСТЕМ.....	9
1.1 Планирование информационной системы	9
1.2 Отказоустойчивость на инфраструктурном уровне	10
1.2.1 Основа отказоустойчивости.....	10
1.2.2 Режим зеркалирования памяти.....	10
1.2.3 RAID	11
1.2.4 Резервное копирование	16
1.2.5 Георезервирование.....	20
1.2.6 Кластер высокой доступности.....	21
1.3 Отказоустойчивость на архитектурном уровне	24
1.3.1 Основа архитектурной устойчивости	24
1.3.2 Монолит или микросервисная архитектура.....	25
1.3.3 Виртуальные машины и контейнеры	27
1.3.4 Оркестраторы контейнеров.....	29
1.3.5 Масштабирование базы данных.....	31
1.3.6 Балансировка нагрузки.....	33
1.4 Анализ лучших практик построения отказоустойчивых информационных систем	34
1.4.1 DevOps	34
1.4.2 Continuous Integration и Continuous Delivery / Deployment.....	35
1.4.3 Среды развертывания	36

1.4.4	Стратегии развертывания приложений	37
1.4.5	Мониторинг	40
1.4.6	Восстановление после сбоев	40
1.5	Постановка задачи	41
1.6	Вывод по 1 главе	42
ГЛАВА 2. ПОСТРОЕНИЕ И РЕАЛИЗАЦИЯ ОТКАЗОУСТОЙЧИВОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ		43
2.1	Структура проекта	43
2.2	Микросервисы	45
2.2.1	Фронтенд: UI	45
2.2.2	Бэкенд: шифрование и дешифрование ГОСТ Р 34.12-2015	46
2.2.3	Бэкенд: вспомогательные инструменты	46
2.3	Docker образы	47
2.3.1	Микросервис фронтенда	47
2.3.2	Микросервис шифрования и дешифрования	47
2.3.3	Микросервис инструментов	48
2.3.4	Использование хеш-сумм и минимальных базовых образов	48
2.4	Swagger-документация микросервисов	49
2.5	Валидация данных	50
2.6	Дополнение нулями для алгоритма шифрования «Кузнечик»	53
2.7	Краткое описание API микросервисов бэкенда	55
2.8	CI-конвейер в Jenkins	56
2.9	Kubernetes манифесты	59
2.10	Вывод по 2 главе	63

ГЛАВА 3. ТЕСТИРОВАНИЕ РАЗРАБОТАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ	64
3.1 Проверка работоспособности.....	64
3.2 Недоступность микросервиса	65
3.3 Масштабирование микросервисов.....	66
3.4 Обновление системы	67
3.5 Вывод по 3 главе	68
ЗАКЛЮЧЕНИЕ.....	70
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	72

ВВЕДЕНИЕ

В современном мире, где бизнес все больше зависит от IT-технологий, вопросы информационной безопасности приобретают все большую значимость. Информационная безопасность — это различные меры по защите информации от посторонних лиц [1].

В концепции информационной безопасности выделяют три принципа:

- конфиденциальность – доступ к информации есть только у того, кто имеет на это право;
- целостность – информация сохраняется в полном объеме и не изменяется без ведома владельца;
- доступность – тот, кто имеет право на доступ к информации, может ее получить.

Большинство современных компаний ежедневно сталкиваются с задачей гарантировать бесперебойную работу своих сервисов. Сбои и простои могут привести к значительным финансовым потерям, урону репутации, а также к потере доверия клиентов и бизнес-партнеров. Именно поэтому создание надежной IT-инфраструктуры должно стать неотъемлемой частью стратегии любой компании, заботящейся о своем будущем.

Отказоустойчивость IT-инфраструктуры — это способность системы продолжать нормальную работу даже в случае частичных сбоев, неожиданных ошибок или выхода из строя отдельных элементов системы. Это означает, что если даже происходят сбои в отдельных частях IT-системы, то они не влекут за собой остановку всего сервиса в целом. В системе с высокой отказоустойчивостью подобные проблемы выявляются и устраняются автоматически, не требуя вмешательства администратора, и они не оказывают влияния на общую производительность или доступность системы [2].

В данной работе основное внимание будет уделено принципу доступности как важнейшему условию обеспечения отказоустойчивости современных информационных систем.

Объектом исследования являются микросервисные информационные приложения.

Предметом исследования, в свою очередь, является построение отказоустойчивой информационной системы с применением инструментов контейнеризации и оркестрации контейнеров.

Целью научно-исследовательской работы является разработка отказоустойчивой информационной системы, основанной на микросервисной архитектуре, с применением современных архитектурных решений и лучших практик обеспечения отказоустойчивости.

Задачами научно-исследовательской работы являются:

1. Анализ принципов и методов построения отказоустойчивых информационных систем;
2. Анализ лучших практик построения отказоустойчивых информационных систем;
3. Построение и реализация отказоустойчивой информационной системы;
4. Тестирование разработанной информационной системы.

ГЛАВА 1. АНАЛИЗ ПРИНЦИПОВ И МЕТОДОВ ПОСТРОЕНИЯ ОТКАЗОУСТОЙЧИВЫХ ИНФОРМАЦИОННЫХ СИСТЕМ

1.1 Планирование информационной системы

Отказоустойчивая информационная система должна начинаться с планирования инфраструктуры и проработки архитектуры. Также хорошей практикой будет определение уровня обслуживания.

Соглашение об уровне обслуживания (Service-level agreement, SLA) – это соглашение между поставщиком услуг и заказчиком. Уровни обслуживания – однозначные и измеримые показатели, которые поставщик услуг обязуется соблюдать, например: процент доступности сервиса, время восстановления работы оборудования и сервисов или время ответа службы поддержки. SLA – это стандартный документ для телекоммуникационных и IT-компаний, который имеет большое значение для грамотного управления ожиданиями пользователей и создания прочных деловых отношений [3].

Один из показателей SLA, который может быть включен в соглашение это – доступность услуги. Этот показатель измеряет количество времени, в течение которого служба доступна для использования, в процентном соотношении. Услуга не может быть доступна на 100%, но должна стремиться к этому значению. Таблица 1.1 показывает время недоступности, допустимое для каждого уровня доступности. Рассчитать время доступности можно по формуле (1.1), а по формуле (1.2) – время недоступности.

Таблица 1.1 – Время недоступности, допустимое для каждого уровня доступности услуги

Доступность услуги, %	Время простоя в год	Время простоя в месяц (30 дней)
99% - «две девятки»	87.6 ч	7.2 ч
99.9% - «три девятки»	8.76 ч	43.2 мин
99.99% - «четыре девятки»	52.56 мин	4.32 мин
99.999% - «пять девяток»	5.26 мин	25.92 сек
99.9999% - «шесть девяток»	31.54 сек	2.59 сек

$$uptime = total_time * \frac{SLA}{100\%} \quad (1.1)$$

$$downtime = total_time - uptime \quad (1.2)$$

На основе принятых соглашений и должна проектироваться система. Нужно понимать, чем строже будут требования, тем дороже выйдет инфраструктура и выше требуемая квалификация специалистов, способных настроить и поддерживать соответствующую архитектуру системы. Также необходимо учитывать тот факт, что со временем система, как правило, обрastaет новым функционалом, что поспособствует обновлению требований SLA, поэтому важно, чтобы система имела потенциал для масштабирования.

1.2 Отказоустойчивость на инфраструктурном уровне

1.2.1 Основа отказоустойчивости

Основная цель при построении надежных систем – гарантировать непрерывность работы IT-инфраструктуры и минимизировать время простоя, что способствует стабильности бизнес-процессов [2].

Основу отказоустойчивости составляет избыточность – наличие запасных компонентов для всех элементов системы [4].

Современные сервера предоставляют широкие возможности для резервирования оперативной памяти, СХД (система хранения данных), питания, интерфейсов.

1.2.2 Режим зеркалирования памяти

Режим зеркалирования памяти (Memory mirroring mode) – особый режим работы оперативной памяти. В этом режиме осуществляется зеркалирование каналов, то есть каналы разбиваются на пары, и в каждой паре один из каналов становится копией другого. Все банки памяти при этом должны быть сконфигурированы идентично [5].

Работа в этом режиме защищает от однобитовых ошибок или выхода из строя всего модуля памяти.

При работе в режиме зеркалирования памяти одни и те же данные записывают в банки системной и зеркалированной памяти. Считывание данных происходит только из системной памяти. При возникновении ошибок, например, многобитовые ошибки или превышено допустимое количество однобитовых ошибок, в системной памяти происходит переназначение банков. Ранее зеркалированная память назначается системной, а системная – зеркалированной. Это обеспечивает бесперебойную работу сервера за исключением случаев, когда ошибка происходит в одном и том же месте в системной и зеркалированной памяти одновременно, вероятность этого крайне мала.

Конечно, такая организация работы приводит к двукратному уменьшению объема оперативной памяти, что подразумевает за собой двукратное увеличение стоимости, но это плата за надежность. Такая тенденция прослеживается всегда, когда речь о дублировании.

1.2.3 RAID

«Кто владеет информацией, тот владеет миром» - Натан Ротшильд. Эта фраза как нельзя лучше отражает критическую важность надежной работы с данными. Потеря данных способна привести к серьезным финансовым потерям, нарушению бизнес-процессов и падению репутации компании.

Одним из методов защиты данных на уровне инфраструктуры является технология RAID (Redundant Array of Independent Disks) – избыточный массив независимых дисков. Это технология объединения двух и более накопителей в единый логический элемент для увеличения отказоустойчивости и производительности сервера.

Существуют различные уровни RAID, с различными показателями надежности и скорости. Самые распространенные уровни: RAID0, RAID1, RAID5, RAID6, RAID10 (RAID1+0).

RAID0. Данный тип работает по принципу поблочного чередования (striping). Все диски объединяются в массив, а данные разбиваются на

одинаковые блоки, которые по очереди равномерно записываются на все накопители. Благодаря этому производительность возрастает пропорционально количеству дисков. Высокая скорость и объем, достигаются ценой отказоустойчивости, при отказе одного из дисков, все данные массива теряются. Также стоит учитывать тот факт, что итоговая производительность массива определяется по самому медленному из дисков, поэтому лучше использовать устройства с одинаковой скоростью. Схематично RAID0 показан на рисунке 1.1 [6].



Рисунок 1.1 – RAID0

RAID1. В основе данного уровня RAID лежит зеркалирование (mirroring). Массив собирается из двух и более дисков, которые дублируют друг друга. Таким образом создаются пары накопителей с идентичными данными, что повышает отказоустойчивость. Если один из дисков выйдет из строя, данные останутся на его точной копии. При высокой отказоустойчивости RAID1 уступает в емкости и производительности на запись, но производительность на чтение будет повышена. Данные необходимо записывать одновременно на каждую пару дисков, где скорость записи ограничивается скоростью работы медленного диска. Схематично RAID1 показан на рисунке 1.2 [6].

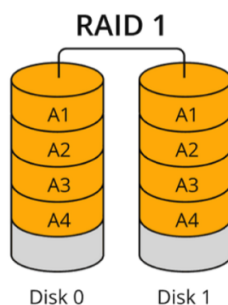


Рисунок 1.2 – RAID1

RAID5. Здесь используется принцип чередования блока данных и распределенной контрольной суммы (четности). Для создания массива нужно как минимум три диска. Блоки данных и контрольные суммы (используется для восстановления недостающих блоков данных в случае потерь) равномерно распределяются по всем дискам.

Под контрольными суммами подразумевается результат операции XOR (исключающее «ИЛИ»). XOR обладает особенностью, которая дает возможность заменить любой операнд результатом, и, применив алгоритм XOR, получить в результате недостающий операнд. Например: $a \text{ xor } b = c$ (где «a», «b», «c» – три диска рейд-массива), в случае если «a» откажет, то его получится восстановить, поставив на его место «c» и проведя хог между «c» и «b»: $c \text{ xor } b = a$ [8].

RAID5 допускает выход из строя одного диска, данные которого можно восстановить, но это займет время. Главное преимущество RAID-5 состоит в самом эффективном использовании дискового пространства из всех отказоустойчивых RAID-массивов. Пригодная для хранения данных емкость дискового пространства составляет от 67% и выше в зависимости от количества дисков в массиве. Схематично RAID5 показан на рисунке 1.3 [6].

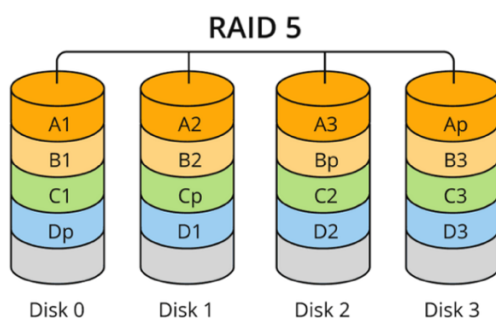


Рисунок 1.3 – RAID5

RAID6. Является развитием RAID5, где используется принцип чередования блока данных и двойной распределенной контрольной суммы (четности). Для создания такого массива потребуется как минимум четыре диска. Здесь аналогично RAID5 блоки данных и контрольные суммы равномерно распределяются по всем дискам, только используются уже два набора контрольных сумм. За счет этого снижается производительность по сравнению с RAID5, но зато обеспечивается лучшая отказоустойчивость. RAID6 допускает выход из строя двух дисков. Пригодная для хранения данных емкость дискового пространства в RAID-6 составляет от 50% и выше в зависимости от количества дисков в массиве [7]. Схематично RAID6 показан на рисунке 1.4 [6].

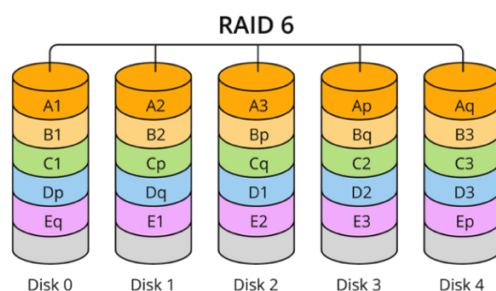


Рисунок 1.4 – RAID6

RAID10. Здесь сочетаются в себе принципы RAID 1 (зеркалирование) и RAID0 (чередование). Для его создания нужно как минимум четыре диска, причем их количество всегда должно быть четным. Массив состоит из нескольких RAID1 массивов, которые затем объединяются в массив RAID0. При такой конфигурации данные остаются целыми, даже если один диск

выйдет из строя или несколько дисков, при условии, что они не относятся к одному зеркалу. При этом, как и в случае с RAID 1, половина общего объема памяти уходит на резервирование. Схематично RAID10 показан на рисунке 1.5 [6].

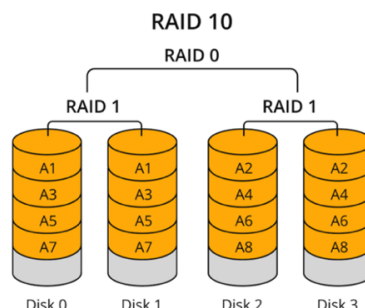


Рисунок 1.5 – RAID10

Для удобства сравнения разных уровней RAID, можно воспользоваться таблицей 1.2, где:

- N – количество дисков в массиве;
- S – объем наименьшего диска.

Таблица 1.2 – Сравнение уровней RAID

Уровень	Кол-во дисков	Эффективная емкость	Допустимое кол-во отказавших дисков	Надежность	Скорость чтения	Скорость записи	Примечание
0	от 2	$S \times N$	-	очень низкая	высокая	высокая	Потеря данных при выходе из строя любого из дисков
1	от 2	S	$N - 1$	очень высокая	средняя	средняя	N -кратная стоимость дискового пространства; максимально возможная надежность; минимально возможный размер, скорость чтения/записи одиночного диска

Продолжение таблицы 1.2

Урове нь	Кол- во диско в	Эффективн ая емкость	Допустим ое кол-во отказавш их дисков	Надежнос ть	Скорос ть чтения	Скорос ть записи	Примечание
5	от 3	$S \times (N - 1)$	1	средняя	высока я	средняя	
6	от 4	$S \times (N - 2)$	2	высокая	высока я	низкая или средняя	скорость записи в зависимости от реализации (может соответствов ать скорости записи RAID 5)
10	от 4, четно е	$S \times \frac{N}{2}$	от 1 до $\frac{N}{2}$	высокая	высока я	высока я	двойная стоимость дискового пространства , потеря данных при выходе из строа группы зеркалирован ия (RAID 1), возможна работа, если хотя бы один любой диск из группы зеркалирован ия (RAID 1) выживает в каждой группе зеркалирован ия (RAID 1)

Каждый уровень RAID имеет свои преимущества и недостатки, поэтому его выбор зависит от приоритетов: скорости, надежности или бюджета.

1.2.4 Резервное копирование

Резервное копирование является важным элементом в обеспечении отказоустойчивости информационных систем. Хорошей практикой при этом является составление плана резервного копирования и аварийного

восстановления, который позволяет быть готовым к восстановлению работоспособности систем и целостности информации в максимально короткие сроки и минимальными потерями для пользователей и бизнеса.

План резервного копирования – это комплекс мер и последовательность действий для создания актуальной копии защищаемых данных на резервном носителе. Основа любого плана это определение целей, поэтому для подготовки в первую очередь необходимо определить целевые показатели для каждого объекта защиты, например, PRO, RTO, BIA, максимально допустимая точка отказа [9].

PRO (Recovery Point Objective, целевая точка восстановления) – это максимальный допустимый интервал времени между последней сохраненной резервной копией и моментом сбоя. Он определяет, какое количество данных может быть потеряно без критических последствий. RPO может быть равен нулю, если система должна обеспечивать «нулевые потери данных».

RTO (Recovery Time Objective, допустимое время восстановления) – это максимально допустимое время, за которое система должна быть восстановлена после инцидента и вновь начать функционировать. Это значение зависит от характера простоя – запланированного, незапланированного или аварийного.

BIA (Business Impact Analysis, анализ воздействия на бизнес) – помогает расставить приоритеты между несколькими объектами защиты в зависимости от размера убытка и степени ущерба вызванного последствиями инцидента.

Максимально допустимая точка восстановления – позволяет определить за какой максимальный период времени данные могут быть восстановлены. Этот параметр влияет на количество и срок хранения резервных копий в хранилище.

Метрики PRO и RTO, показаны на рисунке 1.6, благодаря этой визуализации проще понять, что значит каждая из метрик. Понимание временного диапазона, в котором лежат значения этих показателей, помогает

создать оптимальный план резервного копирования и соотнести цели с затратами для их достижения.

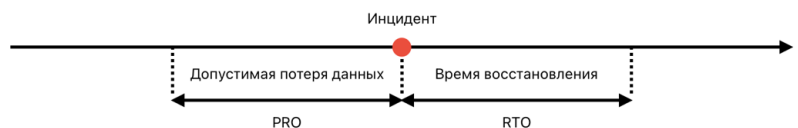


Рисунок 1.6 – Метрики PRO и RTO

В зависимости от потребностей и возможностей, план резервного копирования может включать в себя различные параметры: объект и уровень защиты, приоритеты, расписание, а также вид резервных копий. От того, какой вид копирования используется, зависят объем хранимых данных, скорость создания копий и время восстановления.

Наиболее распространенные виды резервного копирования – это полное, инкрементное и дифференциальное.

Full backup (полное резервное копирование) – вид резервного копирования, при котором создаются копии всей информации, которая требует защиты. Все последующие бэкапы также содержат полный объем копируемых данных. Этот тип копирования данных считается самым надежным, поскольку каждая из копий содержит полный набор информации. Однако, это делает процесс сохранения копий долгим и требует очень много места. Схематично данный вид резервного копирования показан на рисунке 1.7 [10].



Рисунок 1.7 – Схема полного резервного копирования

Incremental backup (инкрементное резервное копирование) – данный вид резервного копирования является альтернативой полному бэкапу. При таком копировании только первая резервная копия содержит весь объем копируемых данных. Все следующие инкрементные копии содержат только данные, которые были изменены после последнего копирования. Этот метод занимает значительно меньше места. Однако, полное восстановление занимает больше времени, так как для этого необходимо восстановление не только полной резервной копии, но и всех инкрементных. Также при потере хотя бы одного инкрементного сохранения, полное восстановление будет невозможно. Схематично данный вид резервного копирования показан на рисунке 1.8 [10].



Рисунок 1.8 – Схема инкрементного резервного копирования

Differential backup (дифференциальное резервное копирование) – в данном виде резервного копирования только первая копия содержит полный объем копируемых данных. Последующие дифференциальные копии содержат те данные, которые были изменены после полного бэкапа. Такой метод позволяет выполнить полное восстановление быстрее, требуя только полной резервной копии и последней дифференциальной резервной копии. Однако, постепенное увеличение размера дифференциальных резервных копий приводит к увеличению занимаемого места и времени копирования. Схематично данный вид резервного копирования показан на рисунке 1.9 [10].



Рисунок 1.9 – Схема дифференциального резервного копирования

Кратное сравнение видов резервного копирования приведено в таблице 1.3.

Таблица 1.3 – Сравнение видов резервного копирования

Вид резервного копирования	Что копируется	Скорость резервного копирования	Требуемое пространство	Скорость восстановления
Full backup	все данные	низкая	высокая	высокая
Incremental backup	только изменения с момента последнего резервного копирования	высокая	низкая	низкая
Differential backup	изменения с момента последнего полного резервного копирования	средняя	средняя	средняя

Совершенно необязательно ограничиваться каким-либо из видов резервного копирования, их можно комбинировать для достижения целей по сохранности данных, времени их восстановления и затратам на их содержание.

1.2.5 Георезервирование

Дублирование каналов связи – избыточность, при которой в случае отказа одного канала сетевое соединение продолжит работу за счет второго, резервного маршрута. Также, например, сервера среднего и старшего уровня имеют по два блока питания, а иногда они оснащаются тремя и более блоками питания. В случае выхода из строя одного из них, сервер продолжит работать от дублирующего блока. Важно помнить, что дублирование не устраняет

возможность сбоев – оно лишь помогает минимизировать их влияние на работу.

Однако остаются риски, связанные с недоступностью всего центра обработки данных (ЦОД). Например, что будет, если в ЦОД пропадет интернет из-за строительных работ, где по ошибке будут перерезаны не те кабели, или отключится электричество из-за какой-то аварии на подстанции?

Важно, чтобы система была устойчива не только к неполадкам в работе программ и оборудования, но и к стихийным бедствиям и серьезным авариям: ураганам, пожарам, отключению электричества и другим. Согласно исследованиям, проведенным в 2021 году институтом Uptime Institute, самая распространенная точка отказа – отключение электричества [4, 11].

Для того, чтобы защититься от такого сценария отказа, используют георезервирование (географическое резервирование). Суть метода заключается в размещении ключевых компонентов системы – таких как серверы приложений, базы данных, хранилища, управляющие узлы и элементы сетевой инфраструктуры – на географически разнесенных площадках. Такой подход позволяет минимизировать риски потери данных и простоев при возникновении сбоев на одной из площадок.

При проектировании георезервирования важно учитывать, что между ЦОД неизбежны сетевые задержки, необходимо решить проблемы синхронизации данных, а также не забывать учитывать требования законодательства в различных регионах [12].

1.2.6 Кластер высокой доступности

Материнская плата – компонент сервера, задублировать который невозможно. Это не единственный компонент, для которого трудно или невозможно организовать избыточность. При выходе из строя таких компонентов сервер теряет свою работоспособность.

Для решения этой проблемы несколько серверов с помощью специального программного обеспечения, объединяются в единую систему.

Такая система называется кластером высокой доступности (high availability cluster, HA-кластер). В случае аварии на одном из серверов, входящий в кластер, его нагрузка перекладывается на другие сервера из этого кластера. Кластеры позволяют не только избежать недоступности систем при выходе оборудования из строя, но и проводить техническое обслуживание или любые другие регламентные работы без остановки ИТ-систем.

Для настройки отказоустойчивого кластера нужно как минимум два узла. Такая конфигурация так и называется – двухузловая. Существуют различные схемы реализации HA-кластеров. Наиболее распространенные из них: Active/Passive, Active/Active, N+1, N+M [13].

Active/Passive. В данной реализации резервируется каждый узел в кластере. Все вычисления производятся на основных серверах, а резервные сервера включаются в работу при выходе из строя основного узла, без потери производительности. Такая конфигурация очень дорогая, так как каждый узел дублируется. Схема Active/Passive HA-кластера показана на рисунке 1.10.

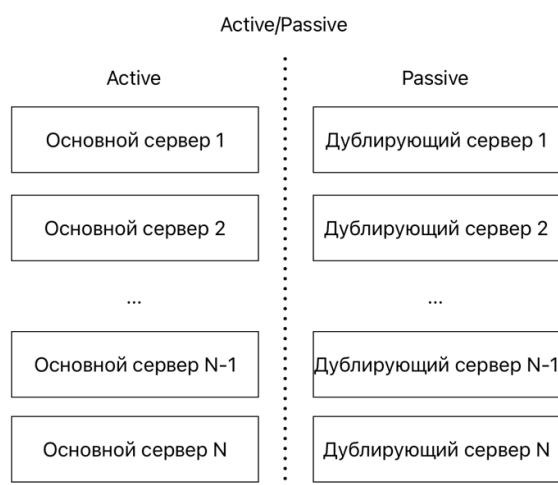


Рисунок 1.10 – Схема Active/Passive HA-кластера

Active/Active. Одна из самых популярных схем реализации отказоустойчивого кластера. Если один из серверов в кластере окажется недоступным, его трафик перераспределится между оставшимися узлами. Если серверов в кластере немного, то это приведет к снижению производительности, так как нагрузка на оставшиеся в кластере сервера

увеличится. Данный подход существенно дешевле полного дублирования узлов. Схема Active/Active HA-кластера показана на рисунке 1.11.

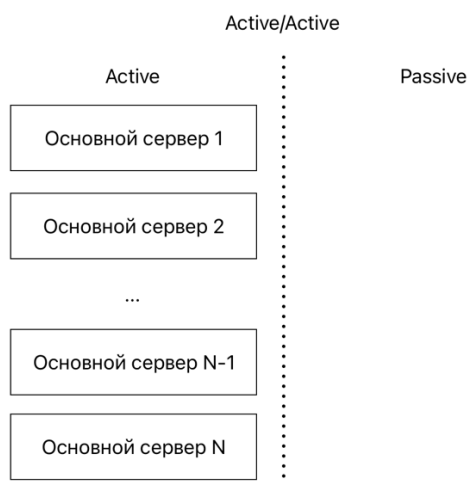


Рисунок 1.11 – Схема Active/Active HA-кластера

N+1. Еще один подход, который позволяет избежать как высоких расходов, так и потери производительности кластера при отказе одного из узлов. В этой конфигурации кластер имеет один полноценный резервный сервер, который при работе в обычном режиме не несет на себе никакой нагрузки, а включается в работу только в случае отказа одного из активных серверов. Схема N+1 HA-кластера показана на рисунке 1.12.

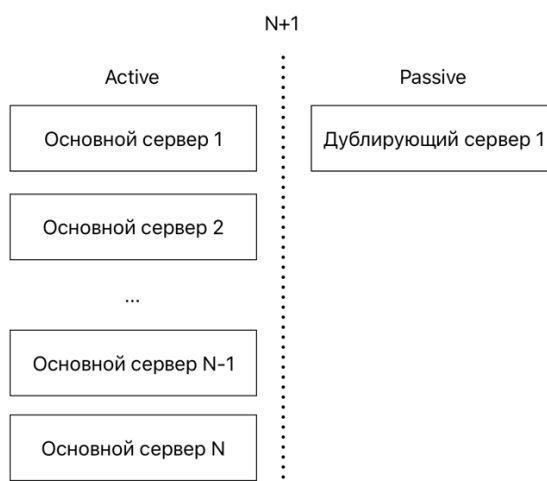


Рисунок 1.12 – Схема N+1 HA-кластера

N+M. Данный подход похож на схему N+1, только вместо одного резервного сервера используется M серверов. Такой способ построения отказоустойчивого кластера позволяет достичь компромисса между

стоимостью решения и требуемым уровнем надежности. Схема N+M HA-кластера показана на рисунке 1.13.

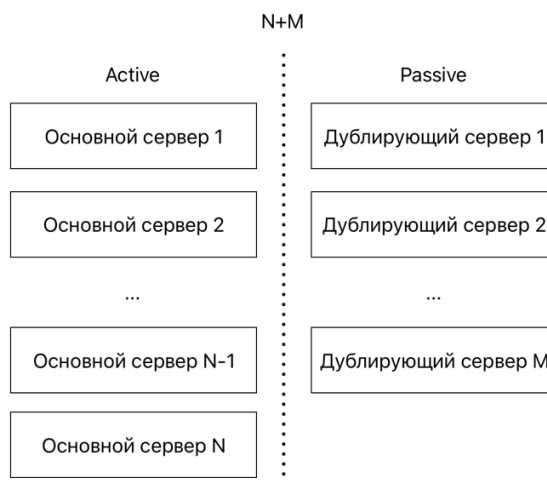


Рисунок 1.13 – Схема N+M HA-кластера

Кратное сравнение конфигураций HA-кластеров приведено в таблице 1.4.

Таблица 1.4 – Сравнение конфигураций HA-кластеров

Конфигурация	Стоимость	Производительность при отказе
Active/Passive	очень высокая (рабочие узлы + такое же число резервных)	сохраняется (резервные узлы берут на себя нагрузку)
Active/Active	низкая (рабочие узлы, все в работе)	снижается (нагрузка перераспределяется между оставшимися узлами)
N+1	средняя (рабочие узлы + один резервный узел)	сохраняется при отказе до одного узла
N+M	высокая (рабочие узлы + M резервных узлов)	сохраняется при отказе до M узлов

1.3 Отказоустойчивость на архитектурном уровне

1.3.1 Основа архитектурной устойчивости

Продуманная, отказоустойчивая инфраструктура – это только часть работы. Следующий этап заключается в том, как спроектировать отказоустойчивое приложение и как и где его запустить.

Современные приложения могут быть реализованы как в виде монолита, так и с использованием микросервисной архитектуры. Взаимодействовать с

приложением можно синхронно через Rest или асинхронно через Kafka. Важную роль играет и то, где приложение будет запущено, оно может работать внутри виртуальных машин или контейнеров, а сами контейнеры – управляться средствами оркестрации, такими как Kubernetes или OpenShift. Кроме того, особое внимание следует уделить масштабированию баз данных.

1.3.2 Монолит или микросервисная архитектура

При проектировании отказоустойчивых информационных систем важным решением становится выбор архитектуры приложения. Существуют две основные архитектуры: монолитная и микросервисная, также известная как микросервисная.

Монолит представляет собой единую систему, в которой все компоненты приложения тесно связаны и разрабатываются как одно целое. Такой подход упрощает начальную разработку, отладку и развертывание, но имеет существенные ограничения при масштабировании и обновлении. Сбой в одной части приложения может повлечь за собой отказ всей системы, а изменения требуют повторной сборки и тестирования всего приложения.

Микросервисная архитектура, напротив, разбивает систему на независимые сервисы, каждый из которых выполняет конкретную задачу и взаимодействует с другими через стандартизированные интерфейсы. Это позволяет изолировать сбои, масштабировать отдельные части приложения, применять разные технологии для разных сервисов. Однако микросервисы предъявляют более высокие требования к организации сети, мониторингу, оркестрации и согласованности данных.

С точки зрения отказоустойчивости, микросервисный подход обеспечивает большую гибкость и устойчивость, но требует продуманной архитектуры и инфраструктуры.

Для сравнения монолитной и микросервисной архитектуры удобно воспользоваться таблицей 1.5 [14].

Таблица 1.5 – Сравнение монолитной и микросервисной архитектуры

Характеристика	Монолитная архитектура	Микросервисная архитектура
Разработка и развертывание	одна база кода; простые процессы разработки и развертывания	децентрализованная разработка; каждый сервис разрабатывается и развертывается по отдельности
Масштабируемость и производительность	масштабируемость всего приложения; возможно нерациональное использование ресурсов	детализированная масштабируемость; оптимальное распределение ресурсов, исходя из потребностей различных сервисов
Обслуживание и расширяемость	единая база кода; проще понять и легче обслуживать	независимые сервисы; проще добавлять новые функции, не изменяя все приложение
Технологическое разнообразие и автономность	ограниченный стек технологий; привязка к определенному набору технологий	разнообразие технологий, доступных для каждого сервиса; можно выбрать, какие технологии подойдут лучше
Отказоустойчивость	единая точка отказа; отказы влияют на все приложение	изолированные отказы; отказоустойчивость лучше
Координация взаимодействия	прямые вызовы функций; более простая коммуникация и обмен данными	взаимодействие между сервисами; дополнительные средства для координации и согласованности данных

Схематично монолитная и микросервисная архитектура показана на рисунке 1.14. На данной схеме видны основные преимущества микросервисного подхода. Так если нагрузка на сервис оплаты возрастет в монолите придется поднимать второй экземпляр приложения, в котором будут сервисы, не нуждающиеся на данный момент в масштабировании. Отсюда и нерациональный подход к использованию ресурсов. В то же время микросервисная архитектура, позволяет увеличить количество экземпляров конкретного нагруженного сервиса.

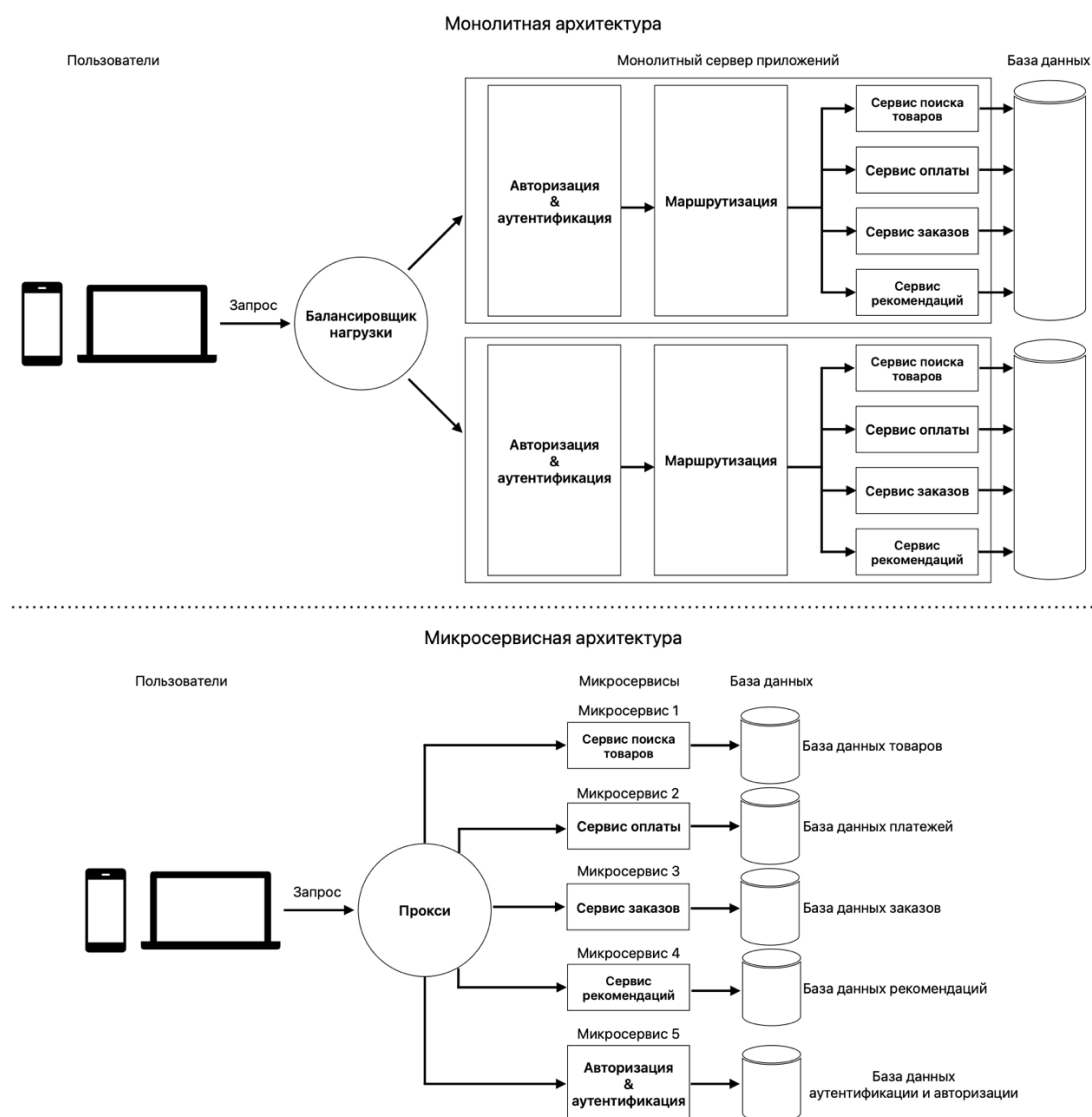


Рисунок 1.14 – Схема монолитной и микросервисной архитектуры

Также не стоит забывать и про другой сценарий, при котором существует только один экземпляр монолита и по одному экземпляру в микросервисах. В случае отказа сервиса оплаты, в монолите будет потеряна работоспособность всего приложения, в то время при использовании микросервисной архитектуры, невозможно будет только совершить оплату, а все остальные функции приложения продолжат свою работу.

1.3.3 Виртуальные машины и контейнеры

Современные отказоустойчивые информационные системы все чаще разворачиваются в изолированных средах – виртуальных машинах (VM) или контейнерах. Обе технологии позволяют добиться изоляции процессов,

гибкости при масштабировании, быстрого восстановления после сбоев и эффективного использования ресурсов.

Виртуальные машины предоставляют полную эмуляцию физического сервера, включая собственную операционную систему. Это делает их универсальными и независимыми от основной ОС хоста, но требует значительных ресурсов и времени на запуск. Виртуализация хорошо подходит для изоляции приложений.

Контейнеры, в отличие от виртуальных машин, используют ядро операционной системы хоста, что делает их легкими, быстрыми и идеальными для микросервисной архитектуры. Контейнеры запускаются практически мгновенно, занимают меньше ресурсов и лучше масштабируются. Их легко интегрировать с системами оркестрации – например, Kubernetes или OpenShift.

Для сравнения виртуальных машин и контейнеров удобно воспользоваться таблицей 1.6 [15].

Таблица 1.6 – Сравнение виртуальных машин и контейнеров

Характеристика	Виртуальная машина	Контейнер
Изоляция процесса	Изоляция процесса на аппаратном уровне	Изоляция процесса на уровне ОС
Операционная система	Каждая виртуальная машина имеет отдельную ОС	Каждый контейнер может совместно использовать ОС
Время загрузки	Загружается в считанные минуты	Загружается в считанные секунды
Занимаемое место	Виртуальные машины занимают несколько ГБ	Контейнеры легкие (КБ / МБ)
Доступность готовых решений	Готовые виртуальные машины трудно найти	Готовые контейнеры легко доступны
Перемещение на новый хост	Виртуальные машины могут перемещаться на новый хост	Контейнеры уничтожаются и воссоздаются, а не перемещаются
Время создания	Создание виртуальной машины занимает относительно больше времени	Контейнеры могут быть созданы в считанные секунды
Использование ресурсов	Требуют больше ресурсов	Требуют меньше ресурсов

Схема, сравнивающая виртуальные машины и контейнеры, показана на рисунке 1.15.

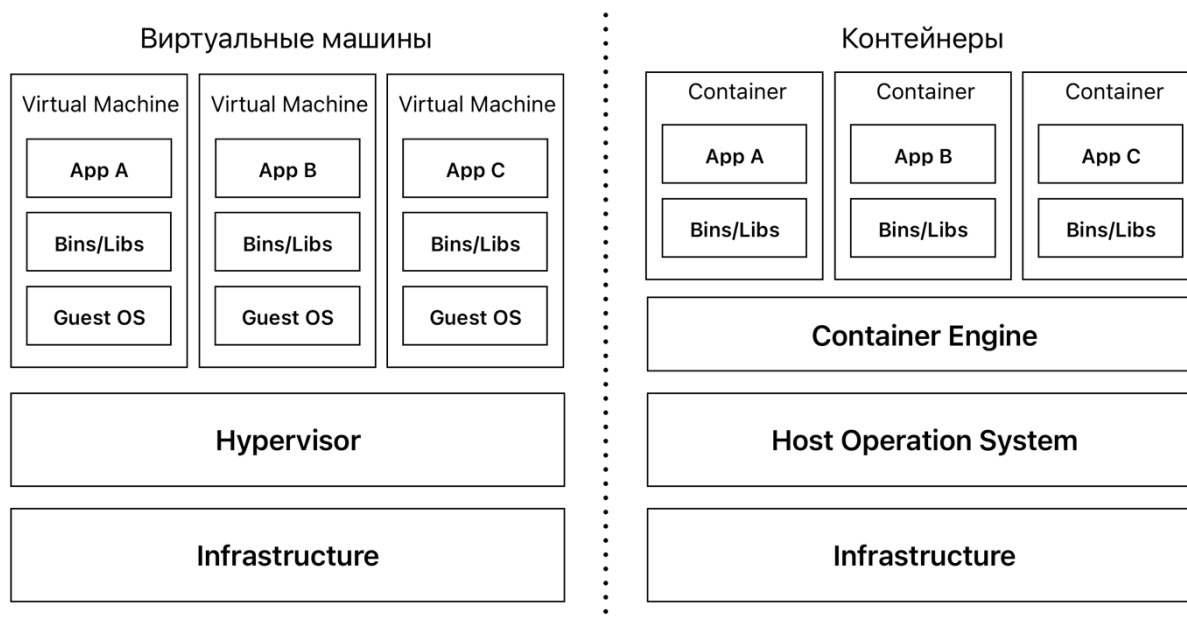


Рисунок 1.15 – Схема виртуальных машин и контейнеров

Провести границу, когда и какую технологию развёртыванию лучше использовать довольно трудно. Часто эти технологии используются вместе. Но тем не менее реализовывать отказоустойчивую информационную систему на основе микросервисов значительно удобнее и эффективнее с помощью контейнеров.

1.3.4 Оркестраторы контейнеров

С развитием контейнеризации появилась необходимость в управлении большим числом контейнеров, их взаимодействием, масштабированием и отказоустойчивостью. Решением этой задачи стали оркестраторы контейнеров – инструменты, которые автоматизируют развёртывание, управление, масштабирование и восстановление контейнеризированных приложений.

Одним из наиболее популярных оркестраторов является Kubernetes (K8s), который предоставляет полный набор механизмов для построения отказоустойчивых систем.

Основные преимущества Kubernetes:

- Автоматическое восстановление. Kubernetes автоматически перезапускает поды, которые перестали работать, перемещает поды на рабочие ноды, убирает и заменяет поды, не прошедшие проверку состояния.

- Масштабируемость. K8s позволяет легко масштабировать приложения как вертикально (добавляя больше ресурсов для каждого отдельного контейнера), так и горизонтально (добавляя больше экземпляров подов).

- Автоматизация развертывания и управления. Kubernetes предоставляет ряд API и механик работы отдельных ресурсов, которые позволяют построить инструменты для автоматического развертывания, обновления и отката приложений. С их помощью можно интегрировать процессы CI/CD и обеспечить быстрое и надежное обновление приложений без простоев.

- Безопасность. Kubernetes обеспечивает изоляцию приложений на уровне контейнеров, а также изоляцию от слоя управления кластером.

- Балансировка нагрузки. Kubernetes автоматически распределяет трафик между подами, обеспечивая высокую доступность и оптимальную производительность приложения.

- Экономия ресурсов. K8s управляет легковесными контейнерами. Это позволяет эффективно использовать вычислительные ресурсы и снижает затраты на инфраструктуру.

- Поддержка микросервисной архитектуры. Kubernetes позволяет управлять множеством небольших, независимо разрабатываемых и развертываемых сервисов. Это позволяет эффективно масштабировать систему, стандартизировать подходы, упрощать обновление кода и продуктов, а также обеспечивать легкость отката изменений [16].

Kubernetes не единственный оркестратор – существует также Red Hat OpenShift, который построен на базе Kubernetes. Kubernetes и OpenShift сегодня являются двумя ведущими инструментами оркестрации контейнеров на рынке. Kubernetes – проект с открытым исходным кодом, в то время как OpenShift представляет собой коммерческое корпоративное решение. Благодаря открытому исходному коду Kubernetes является более предпочтительным вариантом для многих задач.

Оркестраторы контейнеров являются неотъемлемой частью современной архитектуры отказоустойчивых систем, обеспечивая высокий уровень автоматизации и наблюдаемости (observability). Они обеспечивают приложениям устойчивость к сбоям и изменяющимся нагрузкам, а также поддерживают самовосстановление и непрерывную работу в любых условиях.

1.3.5 Масштабирование базы данных

База данных (БД), как и любой компонент системы, должна масштабироваться для повышения отказоустойчивости. Горизонтальное масштабирование (добавление новых серверов или узлов) для БД по умолчанию недоступно, так как это может привести к нарушению консистентности данных. Но существуют различные техники масштабирования, которые открывают такую возможность, например репликация или шардирование.

Техника репликации заключается в создании и поддержании нескольких копий данных на разных серверах или узлах. Эта техника обеспечивает высокую доступность и отказоустойчивость базы данных. В типичной модели репликации «primary-replica» один узел назначается primary (лидер), а остальные становятся replica (копиями):

- лидер обрабатывает все операции записи, обеспечивая их согласованность и целостность. При изменении или добавлении данных в базу лидера, эти изменения автоматически распространяются на узлы-реплики;
- лидер также может обрабатывать критические операции чтения, где требуется высокая степень согласованности. Реплики обычно используются для обработки запросов на чтение, чтобы распределить нагрузку и улучшить производительность системы;
- если лидер выйдет из строя, то одна из реплик станет лидером.

Схема репликации показана на рисунке 1.16. Преимуществами данного подхода является улучшенная производительность на чтение и высокая доступность, даже в случае сбоя на некоторых узлах. А недостатками данного

подхода является задержка в синхронизации данных и сложность реализации [17].

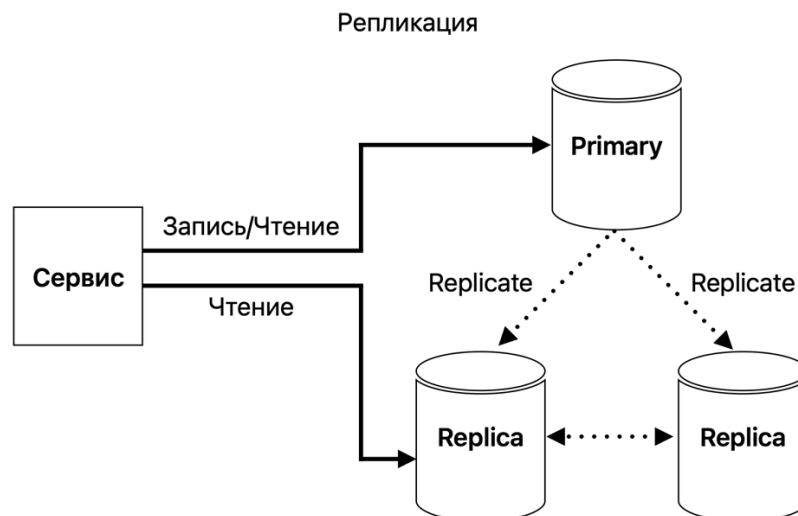


Рисунок 1.16 – Схема репликации базы данных

При использовании техники шардирования одна большая база данных разделяется на более мелкие и управляемые БД, которые называются шардами. Основные стратегии разделения базы:

- шардирование по диапазону значений – данные распределяются между шардами по диапазонам ключей (например, ID от 1 до 1000 – в одном шарде, от 1001 до 2000 – в другом);
- Хеш-шардирование – к значению ключа применяется хеш-функция, результат которой определяет, в каком шарде будут храниться данные;
- Директорное шардирование – используется отдельная таблица-сопоставление, которая указывает, где именно находится каждая запись.

Схема шардирования на основе диапазона ключей показана на рисунке 1.17. Преимущество шардирования заключается в значительном повышении производительности при запросах и операциях записи, так как они могут обрабатываться параллельно, если направлены в разные шарды. К недостаткам относят перебалансировку данных между шардами, которая может быть сложной и времязатратной процедурой, и объединение данных между шардами, которая может стать нетривиальной задачей [17].

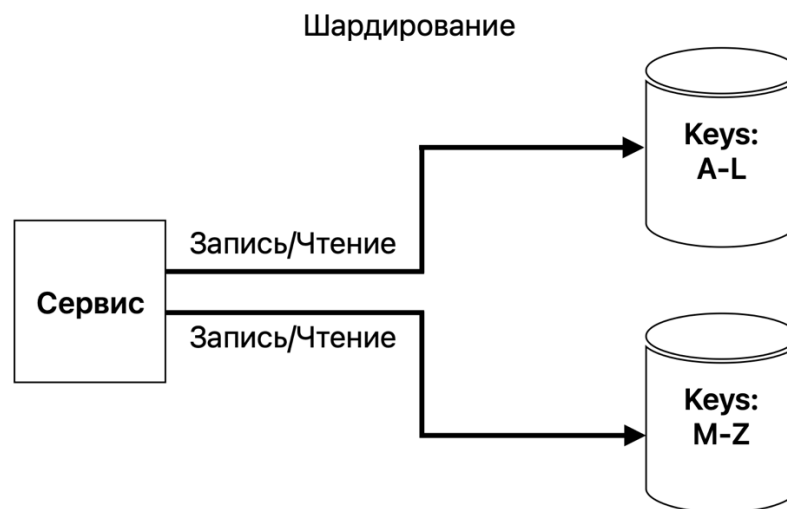


Рисунок 1.17 – Схема шардирования базы данных на основе диапазона ключей

1.3.6 Балансировка нагрузки

При масштабировании информационных систем по горизонтали – будь то отдельные серверы или микросервисы – критически важно обеспечить корректное распределение входящего трафика. Без этого новые ресурсы не будут эффективно использоваться. Решением этой задачи занимаются сервисы балансировки нагрузки, например, Nginx.

Балансировка нагрузки – это распределение входящего сетевого трафика по нескольким серверам или сервисам. Распределение нагрузки обеспечивает стабильную, масштабируемую и отказоустойчивую работу системы, исключая ситуации перегрузки отдельных компонентов.

Балансировщик выступает в качестве промежуточного звена между клиентами и серверами, его задача состоит в том, чтобы принимать входящие запросы от клиентов и перенаправлять их на один из доступных серверов в зависимости от выбранной стратегии распределения нагрузки. Основная цель – обеспечить, чтобы ни один сервер не был перегружен, так это может привести к снижению производительности или даже к отказу в обслуживании.

Среди стратегий балансировки нагрузки распространены следующие:

- Round-robin – запросы поочередно распределяются между всеми доступными серверами.

- Least-connections – запросы отправляются на сервер с наименьшей текущей нагрузкой (минимальное количество открытых соединений).

- IP-hash – балансировщик нагрузки вычисляет хэш от IP-адреса клиента и на основании этого хэша определяет, на какой сервер направить запрос.

- URL-hash – подход похожий на IP-hash, только хэш вычисляется на основе URL-запроса.

Применение балансировки нагрузки не только распределяет нагрузку и улучшает общую производительность, но также повышает общую отказоустойчивость системы. В случае сбоя одного из серверов, балансировщик автоматически исключает его из пула и перенаправляет трафик на оставшиеся узлы, обеспечивая непрерывность обслуживания пользователей. Таким образом, балансировка нагрузки является неотъемлемой частью архитектуры отказоустойчивых систем.

1.4 Анализ лучших практик построения отказоустойчивых информационных систем

1.4.1 DevOps

Основной подход в DevOps – это автоматизация разработки, тестирования и развертки приложений. Благодаря ей, команда может ускорить и стандартизировать все процессы и создать цикл непрерывной интеграции (Continuous Integration) и непрерывного развертывания (Continuous Deployment) приложений – CI/CD-конвейер [18].

Автоматизация позволяет уменьшить ошибки из-за человеческого фактора до минимума. К тому же, благодаря грамотно настроенному CI/CD конвейеру можно увеличить отказоустойчивость приложения, за счет быстрой доставки кода, на различные среды тестирования.

1.4.2 Continuous Integration и Continuous Delivery / Deployment

CI/CD-конвейер предназначен для автоматизации всех этапов разработки и развертывания, что позволяет сокращать время выхода новых версий, снижать вероятность ошибок и повышать стабильность и предсказуемость поставки программного продукта.

Если разделить CI/CD на составляющие, то получится следующие:

- CI (Continuous Integration) – непрерывная интеграция, представляет собой процесс, где написанный код автоматически собирается и проходит unit тестирование, это позволяет выявить возможные ошибки на ранних стадиях;
- CD (Continuous Delivery или Deployment) – непрерывная доставка или развертывание, представляет собой доставку собранного приложения на различные стенды, включая продакшен среду. Отличие delivery от deployment заключается в том, что в delivery релизы новых версий продукта начинается при нажатии на кнопку, в то время как в deployment релиз проходит автоматически.

Схема CI/CD-конвейера показана на рисунке 1.18. Данные конвейер, это не единственно верная реализация. Для каждого проекта конвейер может быть свой, он может включать в себя дополнительные этапы, как в части CI, так и в части CD. Выбор между delivery и deployment, также лучше делать с учетом специфики проекта.

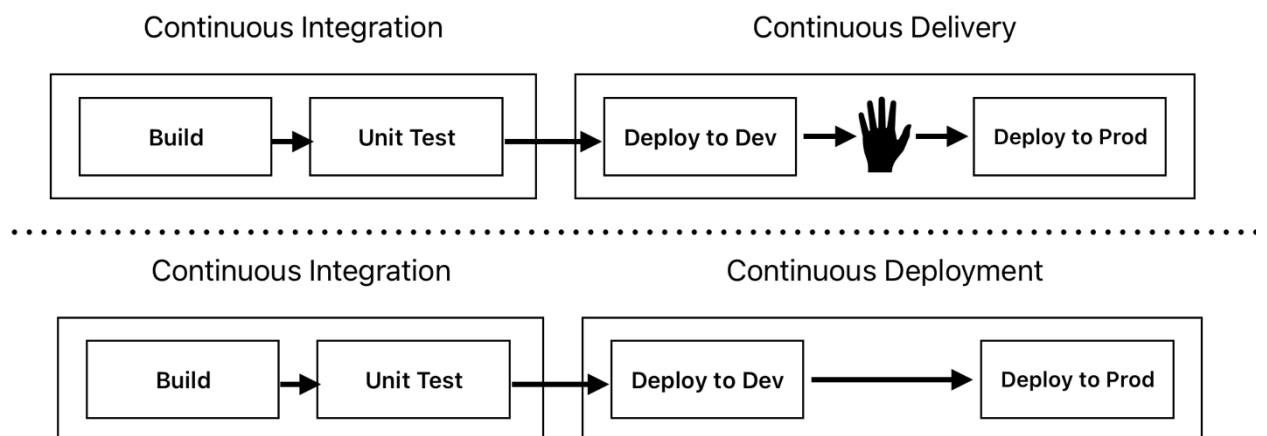


Рисунок 1.18 – Схема CI/CD конвейера

Пример CI части из реального проекта показан на рисунке 1.19. Данный конвейер реализован в Jenkins. Помимо сборки и тестирования проекта, здесь применяются практики DevSecOps, включающие сканирование кодовой базы на наличие уязвимостей с использованием SAST (Статический анализ безопасности приложений, Static Application Security Testing) и SCA (Анализ состава программного обеспечения, Software Composition Analysis), собирается Docker образ и публикуется в registry.

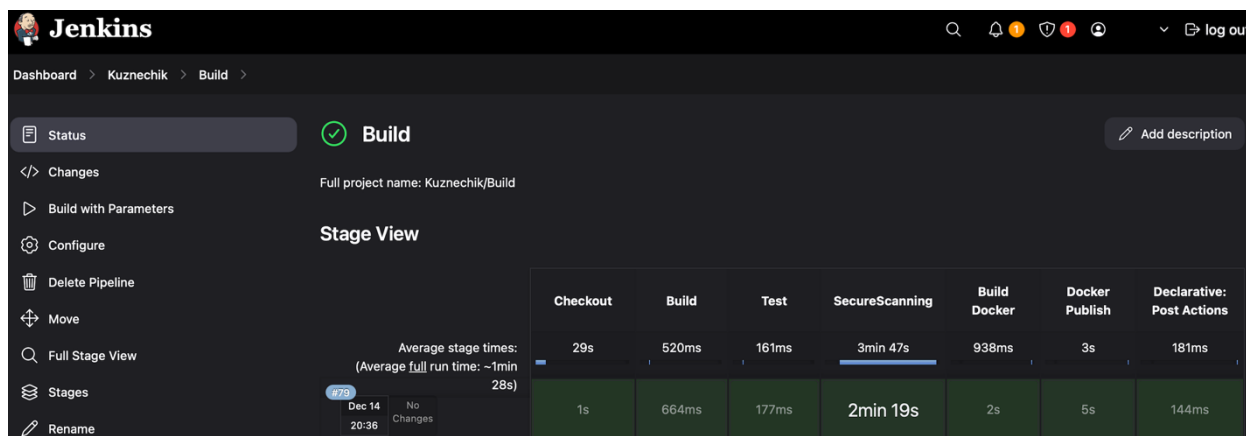


Рисунок 1.19 – CI-конвейер в Jenkins

1.4.3 Среды развертывания

Для повышения отказоустойчивости системы приложение должно быть тщательно протестировано. Для этого выделяю различные среды развертывания. CI/CD-конвейер значительно ускоряет и упрощает развертывание различных версий приложений, однако каждую из сред необходимо поддерживать и настраивать, что требует дополнительных затрат.

Чаще всего распространены следующие среды развертывания:

- Dev (Development) – среда разработки, как правило, здесь проходят первые этапы тестирования с замканными сервисами;
- Test (Testing) – среда тестирования, здесь проверяется корректность интеграционного взаимодействия с другими сервисами;
- Stage (Staging) – среда максимально приближенная к продакшену, на которой проходит финальный этап тестирования приложения;

– Prod (Production) – продакшен среда, на которой работают пользователи приложения.

Сред развертывания может быть, как больше, так и меньше и использоваться они могут с разной целью.

1.4.4 Стратегии развертывания приложений

Как бы хорошо приложение не было бы протестировано, всегда есть вероятность, что при релизе новой версии продукта, произойдет сбой и приложение упадет с ошибкой. Также бывают ситуации необходимости проверки продуктовых гипотез. Для того, чтобы минимизировать вероятность отказа при деплои новой версии приложения или реализовать проверку гипотезы, существуют различные стратегии развертывания приложений.

Стратегии деплоя или развертывания определяют, как будут запускаться приложения/сервисы [19]. В проекте может использоваться как одна стратегия, так и применяться сразу несколько. Распространены следующие стратегии: Recreate, Rolling, Blue/Green, Canary, A/B-тестирование.

Recreate (Повторное создание). Стратегия состоит из двух шагов: удаление текущей версии приложения и развертывание новой. Это самая простая стратегия. Если возникнет какая-то ошибка, то придется откатиться к предыдущей версии, а это бывает не всегда просто сделать. Стратегия развертывания Recreate показана на рисунке 1.20.

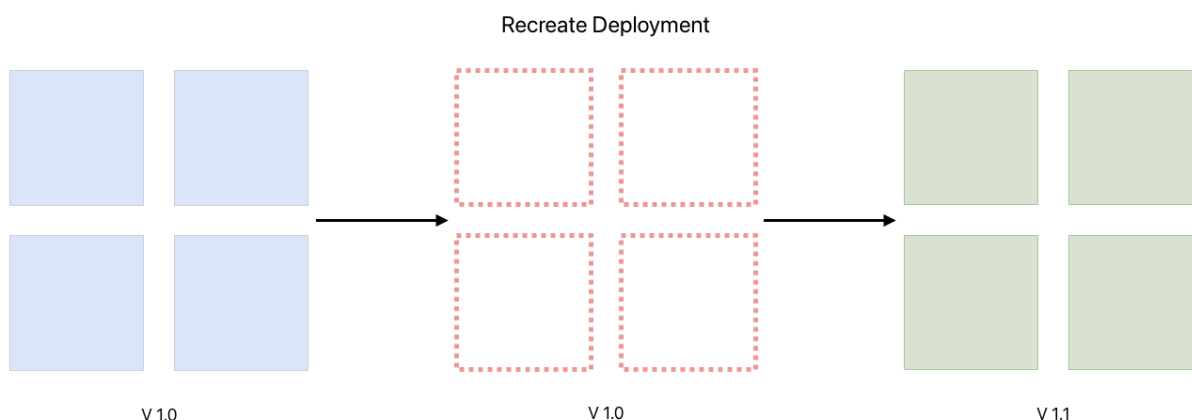


Рисунок 1.20 – Стратегия развертывания Recreate

Rolling (Постепенное развертывание). В этой стратегии все экземпляры приложения будут последовательно обновляться до новой версии. Каждый экземпляр проходит следующий цикл: удаляется один из экземпляров со старой версией приложения (остальные продолжают работать), далее запускается экземпляр с новой версией, затем проверяется корректный запуск нового экземпляра и если все хорошо, то цикл повторяется для следующего экземпляра. Стратегия развертывания Rolling показана на рисунке 1.21.

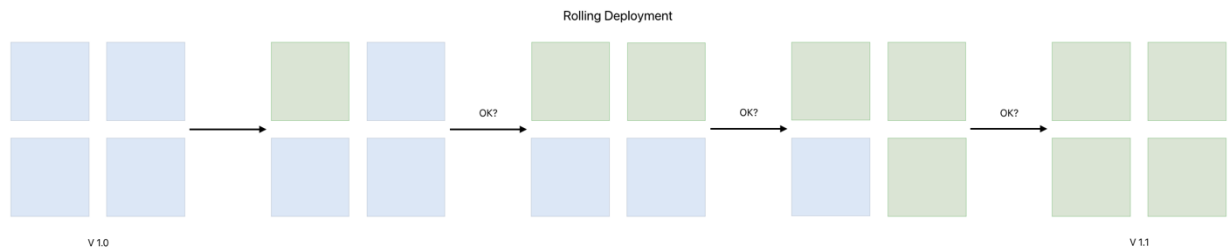


Рисунок 1.21 – Стратегия развертывания Rolling

Blue/Green (Сине-зеленое развертывание). Эта стратегия базируется на двух продуктивных средах: синей и зеленой. В синей среде находится старая версия приложения, а в зеленой – будет запущена новая версия. Когда зеленая среда готова, после тщательного тестирования, принять на себя пользователей, на нее переключается пользовательский трафик. Если с новой версией в зеленой среде возникают проблемы, то трафик возвращается на старую версию приложения, то есть в синюю среду. Стратегия развертывания Blue/Green показана на рисунке 1.22.

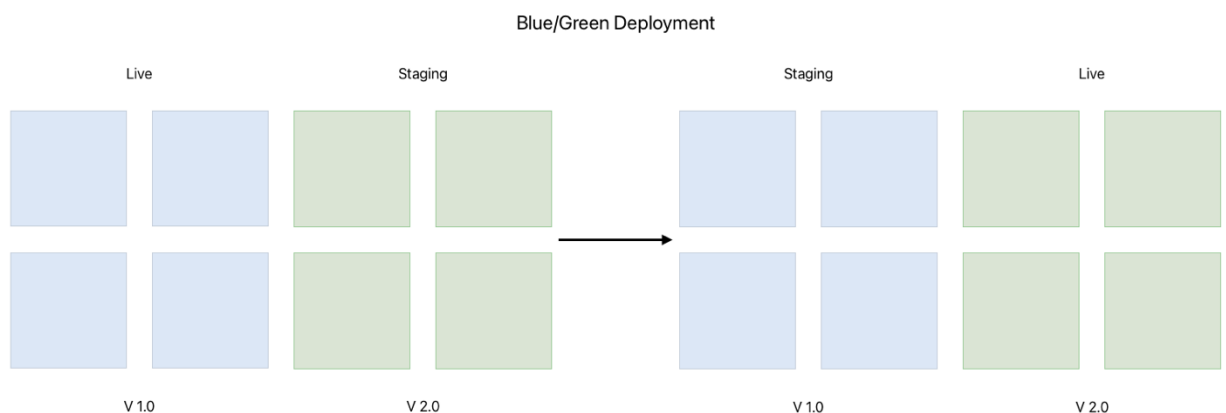


Рисунок 1.22 – Стратегия развертывания Blue/Green

Canary (Канареечное развертывание). Канареечный деплой схож с синей-зеленой стратегией. Отличие в том, что пользователи приложения постепенно переключаются на новую версию. Часть текущих экземпляров приложения заменяется новой версией, на которую переключается часть трафика. Если новая версия работает стабильно, то доля пользователей, использующих новую версию приложения, постепенно увеличивается. Так происходит до тех пор, пока все неактуальные экземпляры не будут заменены. Если же в процессе обновления возникают проблемы, то трафик можно быстро вернуть на предыдущую версию. Стратегия развертывания Canary показана на рисунке 1.23.

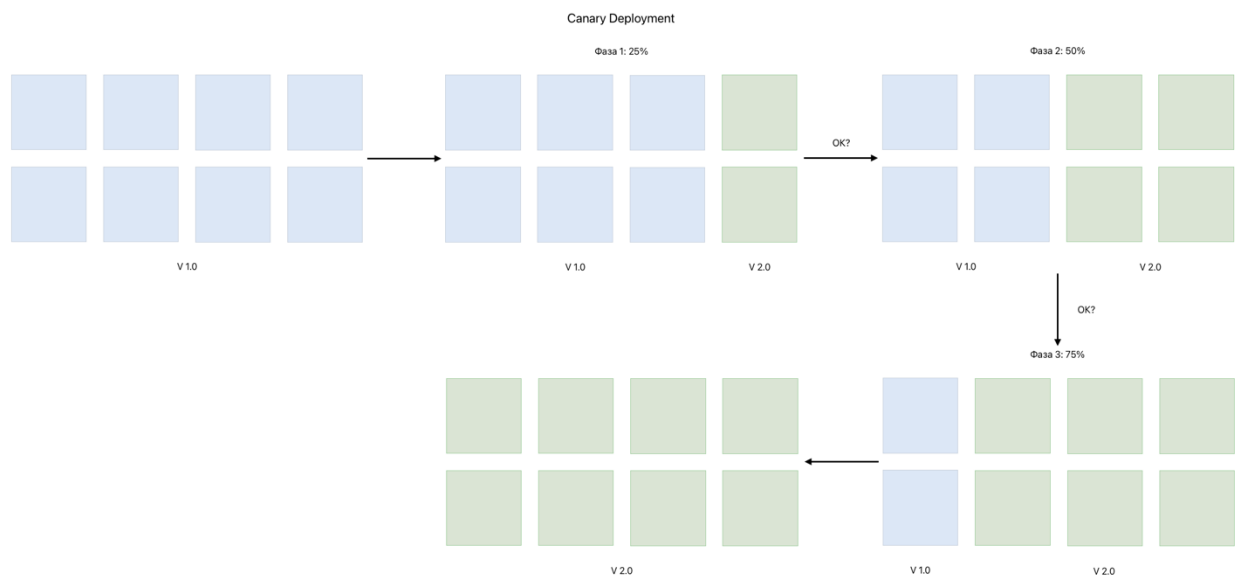


Рисунок 1.23 – Стратегия развертывания Canary

А/В-тестирование. Фактически это вариация канареечного развертывания применительно для фронтенда. Здесь также запускается ограниченное количество экземпляров с новой версией приложения, на которую переключается часть пользовательского трафика. Эта стратегия используется больше в экспериментах, для проверки идей и гипотез. Например, с помощью такой стратегии релиза можно узнать, как пользователи реагируют на те или иные изменения. Стратегия А/В-тестирования при развертывании показана на рисунке 1.24.

Восстановление после сбоев (Disaster Recovery, DR) – это стратегия, которая включает в себя процедуры и политики для восстановления критических функций IT-системы после возникновения сбоя или катастрофы [2].

Цель DR – минимизировать время простоя и предотвратить потерю данных. Весь процесс DR подразделяется на четыре этапа:

- оценка рисков и воздействия;
- разработка DR-плана;
- тестирование плана;
- реализация плана после возникновения сбоя.

В плане восстановления после сбоя описываются действия, которые нужно предпринять, а также объясняется, как эти действия будут поддерживать бизнес-процессы. Важно понимать, что не существует универсального идеального плана. План уникален для каждой системы, так как он учитывает всю ее специфику и особенности.

1.5 Постановка задачи

Целью научно-исследовательской работы является разработка отказоустойчивой информационной системы, основанной на микросервисной архитектуре, с применением современных архитектурных решений и лучших практик обеспечения отказоустойчивости.

Разрабатываемая система – это приложение с блочным шифрованием ГОСТ Р 34.12–2015 («Кузнечик») на C++ и взаимодействием по REST API на Python [20]. Эффективность решения будет проверена с помощью тестирования.

Для разработки отказоустойчивой информационной системы необходимо решить следующие задачи:

1. Спроектировать архитектуру приложения;
2. Реализовать блочный алгоритм шифрования ГОСТ Р 34.12–2015 («Кузнечик») на C++;

3. Реализовать веб-сервер с использованием FastAPI на Python;
4. Реализовать Docker образы для каждого микросервиса;
5. Реализовать CI-конвейер для автоматизации сборки и публикации Docker образов микросервисов;
6. Реализовать Kubernetes манифесты;
7. Развернуть приложение в Kubernetes;
8. Провести тестирование разработанной информационной системы.

1.6 Вывод по 1 главе

В данной главе научно-исследовательской работы были проанализированы принципы и методы построения отказоустойчивых информационных систем на инфраструктурном и архитектурном уровне, а также проанализированы лучшие практики.

При проектировании отказоустойчивой системы важно понимать, что с повышением уровня отказоустойчивости возрастает и стоимость реализации. Так дублирование компонентов, как правило, приводит к росту расходов без пропорционального повышения производительности. В то же время на архитектурном уровне, чем сложнее и надежнее система, тем выше требуемая квалификация сотрудников, способных эту систему настроить и поддерживать. Поэтому важно, чтобы затраты на систему были оправданы. Необходимо найти баланс между уровнем отказоустойчивости и реальными потребностями и возможностями бизнеса.

ГЛАВА 2. ПОСТРОЕНИЕ И РЕАЛИЗАЦИЯ ОТКАЗОУСТОЙЧИВОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1 Структура проекта

Проект состоит из двух частей: фронтенда и бэкенда. В каждой из этих частей есть свои микросервисы, так в части фронтенда есть микросервис с UI, позволяющий взаимодействовать с бэкендом, а в бэкенде есть два микросервиса, реализующих функционал приложения.

Для каждого микросервиса реализован свой Dockerfile, который упрощает развертывание приложения. Также для микросервисов написаны Kubernetes манифесты deployment и service, которые необходимы для развертывания приложения в среде k8s.

Сборка образов проходит в Jenkins. Для этого реализован CI-пайплайн, который собирает и публикует образы контейнеров в удаленный реестр образов (registry). Также один из этапов конвейера включает в себя проверку кода на уязвимости, что способствует своевременному их выявлению и устранению.

Для управления исходным кодом всех компонентов проекта используется система Git, позволяющая отслеживать изменения и восстанавливать любые версии кода на разных этапах разработки.

Для отображения файловой структуры проекта можно воспользоваться утилитой tree. Вывод утилиты показан на рисунке 2.1.



Рисунок 2.1 – Файловая структура проекта

2.2 Микросервисы

2.2.1 Фронтенд: UI

Данный микросервис называется «frontend». Фронтенд реализован в HTML, SCSS, JavaScript. При написании HTML-страницы использовалась методология БЭМ, стили страницы описаны в SCSS, который в дальнейшем компилируется в CSS. Использование препроцессора SCSS значительно упрощает работу со стилями и адаптацию страницы под различные версии браузеров, благодаря автоматическому добавлению вендорных префиксов. JS применяется для взаимодействия с бэкендом, чтобы передать и забрать результат работы сервиса.

Эти статические ресурсы фронтенда отдает Nginx сервер. Страница, с которой взаимодействует пользователь, показана на рисунке 2.2.

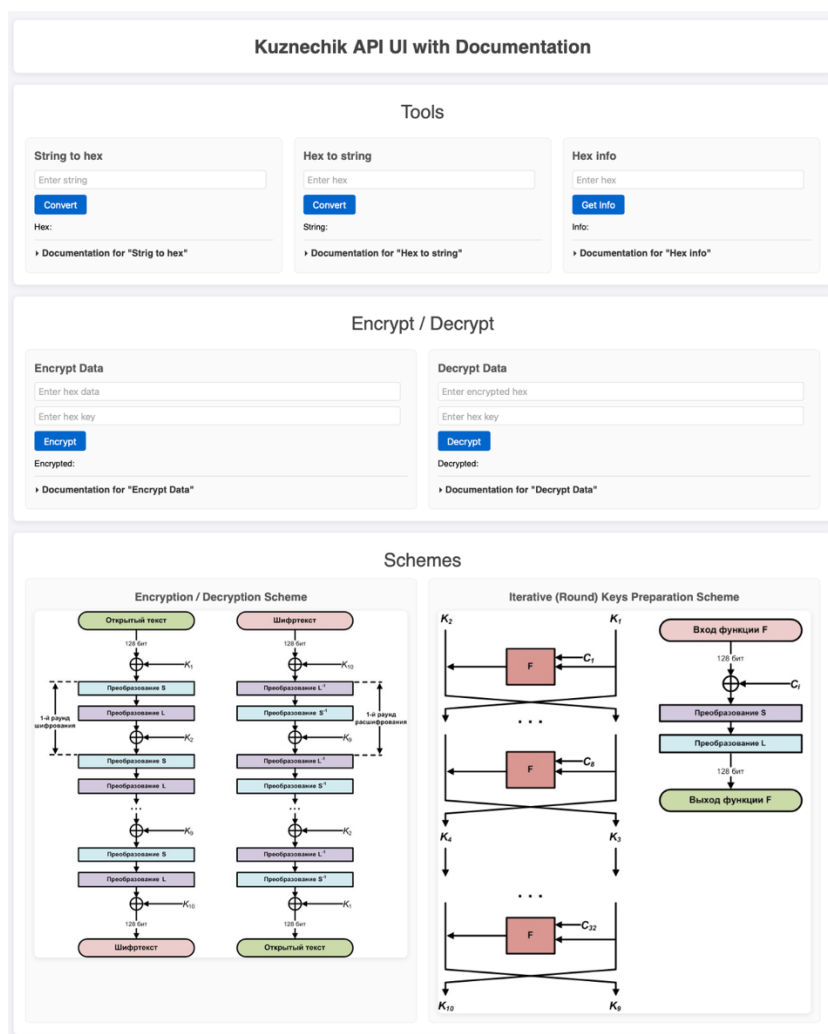


Рисунок 2.2 – UI часть проекта

2.2.2 Бэкенд: шифрование и дешифрование ГОСТ Р 34.12-2015

Данный микросервис называется «kuznechik_crypto». Микросервис написан на двух языках C++ и Python.

На C++ реализовано шифрование и дешифрование блока данных по алгоритму ГОСТ Р 34.12-2015. Код компилируется в shared library для интеграции взаимодействия с Python.

На Python реализовано API, при помощи веб-фреймворка FastAPI, которое взаимодействует с shared library. Помимо этого, на Python реализована дополнительная конечная точка /health/liveness, используемая Kubernetes для проверки состояния сервиса. Также валидация входных параметров осуществляется на стороне Python.

Перечень всех конечных точек микросервиса «kuznechik_crypto»:

- /health/liveness
- /kuznechik/encrypt
- /kuznechik/decrypt

2.2.3 Бэкенд: вспомогательные инструменты

Данный микросервис называется «tools». Данный микросервис написан на Python и тоже использует веб-фреймворк FastAPI. Сервис предоставляет инструменты кодирования и декодирования строк в hex формат, а также дополнительную информацию по hex данным. Здесь также имеется валидация входных параметров и аналогичная конечная точка /health/liveness для Kubernetes.

Перечень всех конечных точек микросервиса «tools»:

- /health/liveness
- /tools/str-to-hex
- /tools/hex-to-str
- /tools/hex-info

2.3 Docker образы

2.3.1 Микросервис фронтенда

Docker образ данного микросервиса основан на Alpine с Nginx. Для сборки данного образа необходимы файлы фронтенда и конфигурационные файлы nginx сервера.

Конфигурация Nginx реализована модульным образом. Основной конфигурационный файл `nginx.conf` подключает конфигурацию `frontend.conf`, которая содержит конфигурацию веб-сервера с настройками порта и имени сервера, а также определяет базовый маршрут для обслуживания статического контента. Кроме того, через директиву `include` подключаются отдельные файлы с настройками проксирования HTTP-запросов к микросервисам. Каждый сервис представлен отдельным конфигурационным файлом, расположенным в директории `proхu`. Такой подход упрощает сопровождение и обеспечивает масштабируемость системы.

2.3.2 Микросервис шифрования и дешифрования

Образ данного микросервиса собирается при помощи многоэтапной сборки (`multi-stage build`).

Для первого этапа используется базовый образ Alpine. На этом этапе выполняется подготовка и компиляции C++ кода в `shared library`.

Для второго этапа используется образ Alpine python версии 3.10. На этом этапе выполняется подготовка python приложения, а также копируется собранная на предыдущем этапе библиотека.

Использование многоэтапной сборки позволяет не хранить зависимости, которые не будут использоваться в работе приложения, что уменьшает конечный вес образа, а также повышает безопасность системы путем минимизации окружения.

2.3.3 Микросервис инструментов

Docker файл для данного микросервиса похож на Docker файл сервиса шифрования и дешифрования, за тем исключением, что здесь не требуется предварительно компилировать C++ код. Поэтому данный Docker образ собирается в один этап и использует в качестве базового такой же образ Alpine python3.10.

2.3.4 Использование хеш-сумм и минимальных базовых образов

Во всех Docker файлах базовые образы указываются с использованием их sha256-сумм. Такой подход обеспечивает воспроизводимость сборки, так как точно фиксирует используемую версию образа, в отличие от тегов, так, например, тег latest может указывать на разные версии с течением времени. Использование хеш-сумм также повышает безопасность: тег может быть переопределен и указывать на другой, потенциально недоверенный образ, тогда как sha256 однозначно идентифицирует конкретный артефакт.

Также все образы строятся на основе легковесного дистрибутива Linux – Alpine. Для того чтобы узнать конечный вес получившихся образов микросервисов, можно собрать их локально с помощью команды, продемонстрированной в листинге 2.1, а затем посмотреть на их вес командой из листинга 2.2.

Листинг 2.1 – Сборка Docker образа

```
$ docker build -f <path_to_dockerfile> -t <microservice>:<version> .
```

Листинг 2.2 – Список локальных образов

```
$ docker images
```

Собранные локально образы показаны на рисунке 2.3. Вес каждого образа не превышает 100 мегабайт. Небольшой вес образов снижает сетевую нагрузку – это особенно важно при ограниченной пропускной способности, а также при частых обновлениях. Также это снижает затраты на хранение артефактов.

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
kuznechik_crypto	0.0.1	8b58119b091f	6 minutes ago	84MB
tools	0.0.1	fe09a74c0e50	51 minutes ago	78.4MB
frontend	0.0.1	2b341ddfbabd	5 days ago	50MB

Рисунок 2.3 – Локально собранные образы микросервисов

2.4 Swagger-документация микросервисов

Каждый из микросервисов бэкенда независим, что позволяет локально запустить тот или иной сервис. В бэкенде приложений реализован Swagger, который описывает API. Для того чтобы попасть в Swagger, нужно запустить собранный образ и перейти по адресу `http://localhost:<port>/docs`. Это делается с помощью команды из листинга 2.3.

Листинг 2.3 – Локальный запуск контейнера

```
$ docker run -p <port>:8000 -t <microservice>:<version>
```

Swagger для микросервиса «kuznechik_crypto» показан на рисунке 2.4, а для микросервиса «tools» – на рисунке 2.5.

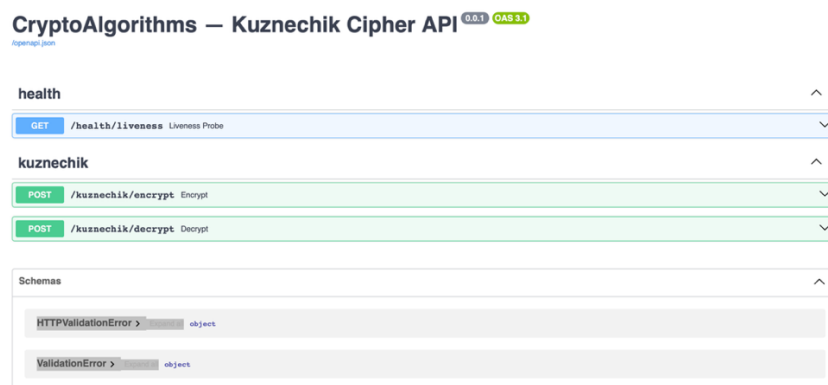


Рисунок 2.4 – Swagger для микросервиса «kuznechik_crypto»

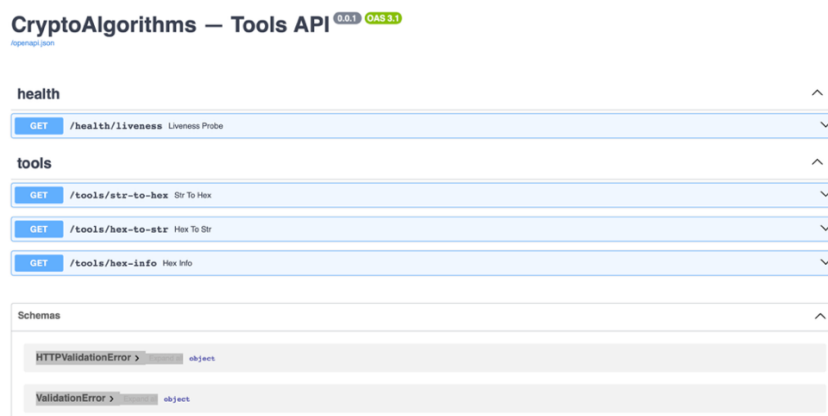


Рисунок 2.5 – Swagger для микросервиса «tools»

Для каждой конечной точки (endpoint) реализована документация, которая позволяет понять, что ожидается на входе, какие есть ограничения и что ожидается на выходе. Так, например, для endpoint `kuznechik/encrypt` документация показана на рисунке 2.6. Также с помощью Swagger можно взаимодействовать с API, отправив запрос через веб-интерфейс, заполнив требуемые параметры.

POST /kuznechik/encrypt Encrypt

Encrypts a 16-byte block of data using the Kuznechik (GOST R 34.12-2015) block cipher.

This endpoint encrypts a block of data (`blk`) using a provided encryption key (`key`). Both the block and key are represented as hexadecimal strings.

Zero padding:

- If the block of data is less than 16 bytes, zero padding is added to the left side of the byte string to match 16 bytes.
- If the encryption key is less than 32 bytes, zero padding is added to the left side of the byte string to match 32 bytes.
- This ensures compliance with the Kuznechik cipher's requirements for fixed block of data and encryption key sizes.

Parameters:

- **blk:** The block of data to be encrypted, represented as a hexadecimal string.
 - Must be no more than 16 bytes (32 hex characters). Zero padding will be applied if shorter.
 - Must contain only valid hexadecimal characters `[0-9a-fA-F]`.
 - Length must be even.
- **key:** The encryption key, represented as a hexadecimal string.
 - Must be no more than 32 bytes (64 hex characters). Zero padding will be applied if shorter.
 - Must contain only valid hexadecimal characters `[0-9a-fA-F]`.
 - Length must be even.

Returns:

- A hexadecimal string representing the encrypted block of exactly 16 bytes (32 hex characters).

Raises:

- **400 Bad Request:**
 - If the `blk` or `key` does not have an even number of characters.
- **422 Unprocessable Entity:**
 - If the input parameters do not meet the validation requirements.

Example:

```

Input:
blk: "48656c66f"
key: "576f726c64"

Output:
"8479704f8d801853d314e7e060f67a80"

Explanation:
- Block of data is zero padded to "00000000000000000000000048656c66f".
- Encryption key is zero padded to "0000000000000000000000000000000000000000000576f726c64".
- Resulting encrypted block is returned as a hexadecimal string.
  
```

Parameters Cancel

Name	Description
blk required string (query)	blk maxLength: 32 pattern: "[0-9a-fA-F]*"
key required string (query)	key maxLength: 64 pattern: "[0-9a-fA-F]*"

Execute

Рисунок 2.6 – Документация и параметры запроса для endpoint `kuznechik/encrypt`

2.5 Валидация данных

Валидация входных параметров обеспечивает отказоустойчивость и повышает защищенность приложения. Все значения, выходящие за пределы допустимого, автоматически отклоняются, что минимизирует вероятность ошибок и ограничивает возможные векторы атак.

Для каждой конечной точки предусмотрена валидация данных. В таблице 2.1 отражены ожидаемые значения входных параметров конечных точек для двух микросервисов бэкенда.

Таблица 2.1 – Endpoints и ожидаемые значения входных параметров для микросервисов бэкенда

Префикс	Endpoint	Параметр	Ожидаемые значения
/health	/liveness		
/tools	/str-to-hex	data_str	– Строка в формате UTF-8 – Длина не должна превышать 32 символа
	/hex-to-str	data_hex	– Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной – Длина не должна превышать 128 символов
	/hex-info	data_hex	– Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной – Длина не должна превышать 128 символов
/kuznechik	/encrypt	blk	– Длина не должна превышать 16 байт (32 шестнадцатеричных символа) – Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной
		key	– Длина не должна превышать 32 байта (64 шестнадцатеричных символа) – Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной
	/decrypt	blk	– Длина ровно 16 байт (32 шестнадцатеричных символа) – Допустимы только шестнадцатеричные символы [0-9a-fA-F]
		key	– Длина не должна превышать 32 байта (64 шестнадцатеричных символа) – Допустимы только шестнадцатеричные символы [0-9a-fA-F] – Длина должна быть четной

В случае, если в endpoint будут переданы данные, которые там не ожидаются, они будут проигнорированы и пользователю будет возвращена ошибка 400 или 422 с соответствующим пояснением, что произошло.

На рисунке 2.7 показана ошибка валидации. Пользователь пытается отправить в микросервис «tools» в endpoint /tools/hex-to-str данные Hello, что недопустимо для данной конечной точки.

```

curl -i 'http://localhost:9000/tools/hex-to-str?data_hex=Hello'
HTTP/1.1 422 Unprocessable Entity
date: Sat, 24 May 2025 18:38:11 GMT
server: uvicorn
content-length: 178
content-type: application/json
{"detail":[{"type":"string_pattern_mismatch","loc":["query","data_hex"],"msg":"String should match pattern '[0-9a-fA-F]+'","input":"Hello","ctx":{"pattern":"^[0-9a-fA-F]+$"]}]}
```

Рисунок 2.7 – Ошибка валидации в endpoint /tools/hex-to-str

На рисунке 2.8 показана та же ошибка, но уже в Swagger.

GET /tools/hex-to-str Hex To Str

Converts a hex string to a regular string.
This endpoint takes a hexadecimal string as input and converts it to a regular UTF-8 string.

Parameters:

- data_hex:** The hexadecimal string to be converted.
 - Must contain only valid hexadecimal characters `[0-9a-fA-F]`.
 - Length must be even and not exceed 128 characters.

Returns:

- A JSON object with the key `data_str`, containing the decoded string.

Raises:

- 400 Bad Request:**
 - If the hex string does not have an even number of characters.
 - If the hex string cannot be decoded into valid UTF-8 encoded text.
- 422 Unprocessable Entity:**
 - If the input hex string exceeds the maximum allowed length (128 characters).
 - If the input hex string contains invalid hexadecimal characters (the string must only contain characters from `[0-9a-fA-F]`).

Example:

```
Input: "48656c66f"
Output: {"data_str": "Hello"}
```

Parameters

Name	Description
data_hex * required	
string	Hello
(query)	maxLength: 128 pattern: "[0-9a-fA-F]+\$"

Please correct the following validation errors and try again.

- For 'data_hex': Value must follow pattern '[0-9a-fA-F]+\$'.

Execute Clear

Рисунок 2.8 – Ошибка валидации в endpoint /tools/hex-to-str

продемонстрированная в Swagger

На рисунке 2.9 показан корректный запрос в endpoint /tools/hex-to-str в Swagger. Пользователь отправляет шестнадцатеричные данные 48656c66f, которые преобразуются в строку UTF-8 со значением Hello.

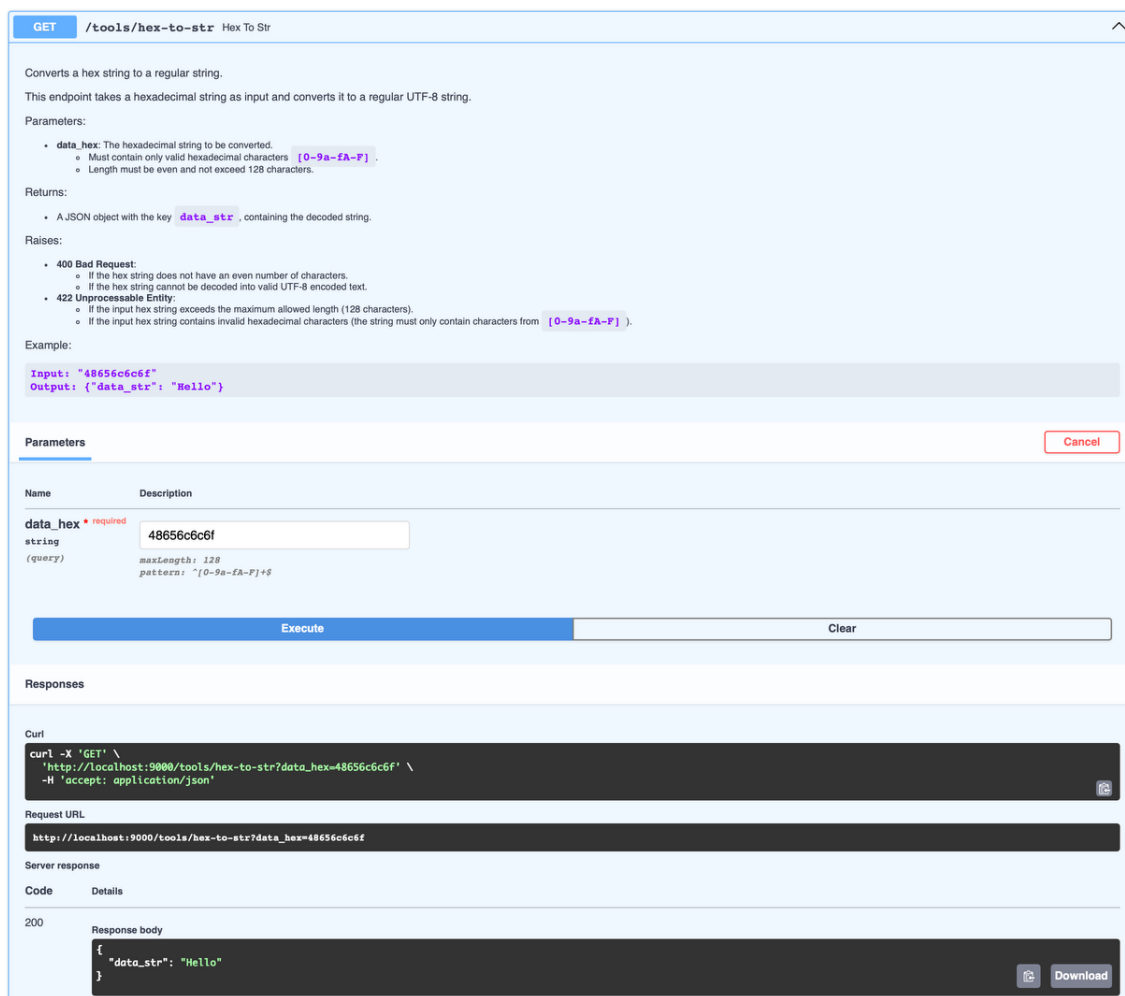


Рисунок 2.9 – Корректный запрос в endpoint /tools/hex-to-str продемонстрированный в Swagger

2.6 Дополнение нулями для алгоритма шифрования «Кузнечик»

Алгоритм «Кузнечик» работает с длиной шифруемого/дешифруемого блока 128 бит (32 шестнадцатеричных числа или 16 байт) и ключом шифрования/дешифрования 256 бит (64 шестнадцатеричных числа или 32 байта). Но не всегда данные соответствуют необходимым размерам. Для случая, когда размер данных меньше требуемого, предусмотрено дополнение нулями (zero padding).

Zero padding дополняет данные нулевыми байтами слева до нужного размера. Рассмотрим на примере, как работает zero padding.

Предположим, пользователь хочет зашифровать слово Hello с помощью ключа World. Представление в шестнадцатеричном формате и длина данных слов показаны в таблице 2.2.

Таблица 2.2 – Информация по словам Hello и World

Слово	Шестнадцатеричное представление	Длина в битах	Длина в шестнадцатеричном формате	Длина в байтах
Hello	48656c6c6f	40	10	5
World	576f726c64	40	10	5

Длина блока данных 40 бит, что меньше требуемых 128, как и длина ключа 40, что меньше требуемых 256 бит. Для решения данной проблемы применяется zero padding, который дополняет шестнадцатеричное представление блока данных и ключа нулевыми байтами слева, до необходимых размеров. Таким образом на вход функции шифрования подаются данные нужного размера. Пример подаваемых данных для слов Hello и World показан в таблице 2.3. В результате шифрования будет получен зашифрованный блок 8479704f8d801853d314e7e060f67a80.

Таблица 2.3 – Информация по словам Hello и World, дополненные нулевыми байтами

Слово	Шестнадцатеричное представление до дополнения нулевыми байтами	Шестнадцатеричное представление после дополнения нулевыми байтами
Hello	48656c6c6f	00000000000000000000000048656c6c6f
World	576f726c64	00000000000000000000000000000000 000000000000000000000000576f726c64

Теперь рассмотрим обратную ситуацию, пользователь хочет дешифровать зашифрованный блок 8479704f8d801853d314e7e060f67a80. На выходе функции шифрования получается зашифрованный блок длиной 128 бит, поэтому при дешифровании zero padding к зашифрованному блоку не применяется, потому что он уже нужного размера. Если же подается блок меньшего размера, то такой блок не пройдет валидацию и пользователю будет возвращена ошибка 422. Для ключа же zero padding применяется по такому же принципу, как и при шифровании. После дешифрования из полученного блока удаляется zero padding (слева удаляются дополненные нулевые байты), и пользователь получает блок данных, который был зашифрован. Таким образом, при дешифровании блока 8479704f8d801853d314e7e060f67a80 ключом 576f726c64 (к ключу применяется zero padding, поэтому в функцию дешифрования передается следующий ключ

[illegible]

2.7 Краткое описание API микросервисов бэкенда

Описание API приложения, как уже отмечалось, предоставлено в Swagger, в таблице 2.1 приведена информация об ожидаемых знамениях параметров. Здесь, в таблице 2.4, будет приведено лишь краткое описание каждой конечной точки для двух микросервисов бэкенда.

Таблица 2.4 – Краткое описание endpoints

Префикс	Endpoint	Краткое описание endpoint
/health	/liveness	Этот endpoint используется для определения, находится ли микросервис в рабочем (живом) состоянии (сервисный endpoint)
/tools	/str-to-hex	Преобразует обычную строку в шестнадцатеричную строку. Этот endpoint принимает на вход строку в формате UTF-8 и преобразует ее в шестнадцатеричное представление.
	/hex-to-str	Преобразует шестнадцатеричную строку в обычную строку. Этот endpoint принимает на вход шестнадцатеричную строку и преобразует ее в строку в формате UTF-8.
	/hex-info	Предоставляет информацию о шестнадцатеричной строке. Этот endpoint принимает на вход шестнадцатеричную строку и возвращает информацию о ее длине в разных единицах измерения (количество символов в шестнадцатеричном формате, байтах и битах).
/kuznechik	/encrypt	Шифрует 16-байтовый блок данных с использованием блочного шифра «Кузнечик» (ГОСТ Р 34.12-2015). Этот endpoint шифрует блок данных (blk) с использованием предоставленного ключа шифрования (key). Блок и ключ представлены в виде шестнадцатеричных строк.
	/decrypt	Расшифровывает 16-байтовый зашифрованный блок с использованием блочного шифра «Кузнечик» (ГОСТ Р 34.12-2015). Этот endpoint расшифровывает зашифрованный блок (blk) с использованием предоставленного ключа для расшифрования (key). Блок и ключ представлены в виде шестнадцатеричных строк.

В таблице 2.5 отображена информация о применении zero padding к входным параметрам endpoints шифрования и дешифрования микросервиса «kuznechik_crypto».

Таблица 2.5 – Применение zero padding в endpoints шифрования и дешифрования микросервиса «kuznechik_crypto»

Префикс	Endpoint	Параметр	Zero padding
/kuznechik	/encrypt	blk	Да
		key	Да
	/decrypt	blk	Нет
		key	Да

Также важно помнить о том, что результат работы endpoint /kuznechik/decrypt возвращается с удаленным zero padding.

2.8 CI-конвейер в Jenkins

В проекте реализован Jenkins пайплайн, позволяющий оптимизировать рутинные задачи в части CI. Пайплайн автоматизирует процессы тестирования кода, проверки кода на безопасность, а также сборку Docker образов и их публикацию в удаленный реестр образов (registry). Схематично пайплайн показан на рисунке 2.10.

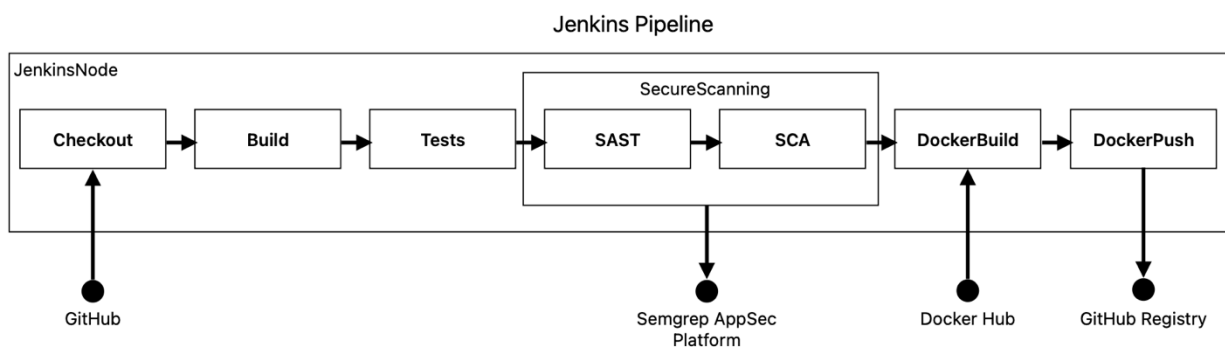


Рисунок 2.10 – Схематичный вид Jenkins пайплайна

На схеме отображены внешние сервисы, с которыми взаимодействует конвейер. Так для этапа «Checkout» должен быть доступ к репозиторию GitHub, для этапа «SecureScanning» должно быть настроено взаимодействие с платформой Semgrep AppSec, для этапа «DockerBuild» требуется доступ к Docker Hub, а для публикации собранных образов, то есть для этапа

«DockerPush», должен быть подготовлен удаленный реестр образов, в данном случае это GitHub Registry.

Безопасность ПО проверяется SAST и SCA анализаторами при помощи платформы Semgrep AppSec Platform. Страница входа на платформу показана на рисунке 2.11.

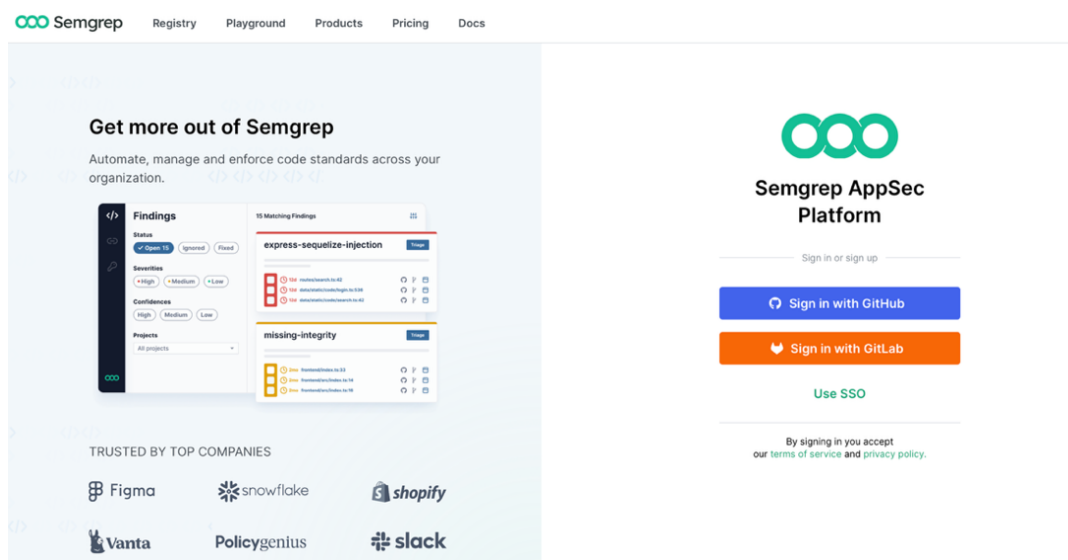


Рисунок 2.11 – Страница входа Semgrep AppSec Platform

Один из критериев хорошего пайплайна – это его адаптивность к изменениям. Добавление новых мультисервисов реализовано крайне просто, необходимо только указать название нового сервиса. Пример текущий конфигурации мультисервисов, Docker образы которых необходимо собрать и опубликовать, показан в линтинге 2.3. Данный конвейер собирает все Docker образы и публикует их параллельно, что позволяет значительно сократить время ожидания.

Листинг 2.3 – Список мультисервисов для сборки и публикации Docker образов

```
SERVICES = "kuznechik_crypto,tools,frontend" // Microservices should be listed separated by ", "
```

Не менее важно иметь возможность управлять запуском работы конвейера. Здесь предусмотрена возможность выбора Jenkins агента, ветки, с которой необходимо собрать проект, можно указать версию контейнеров,

которые будут собраны, а также через булевые параметры, можно передать необходимые этапы сборки. Окно запуска пайплайна показано на рисунке 2.12.

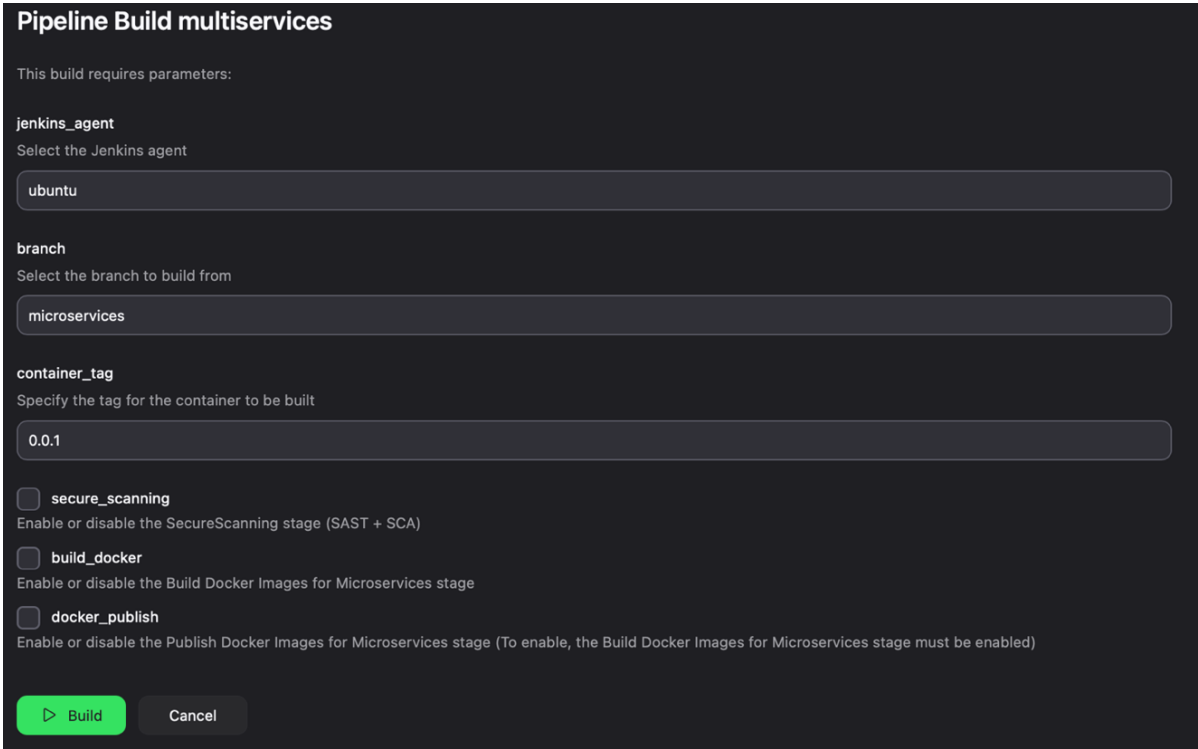


Рисунок 2.12 – Окно запуска пайплайна с выбором необходимых параметров

При каждом запуске всегда будут работать этапы «Checkout», «Build kuznechik_crypto (C++)» и «Test». Все остальные этапы включаются через булевые параметры. На рисунке 2.13 показана успешная работа конвейера со всеми включенными этапами.

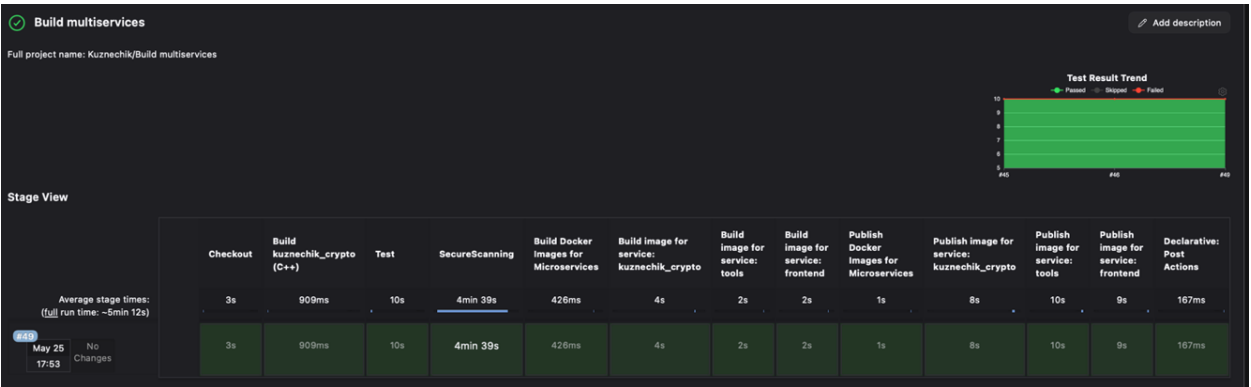


Рисунок 2.13 – Отображение stage в Jenkins UI

Результат сканирования кода на уязвимости, можно посмотреть из логов работы конвейера или в веб интерфейсе Semgrep AppSec Platform.

Опубликованные Docker образы в GitHub Registry показаны на рисунке 2.14. Здесь же можно посмотреть статистику скачивания образов.

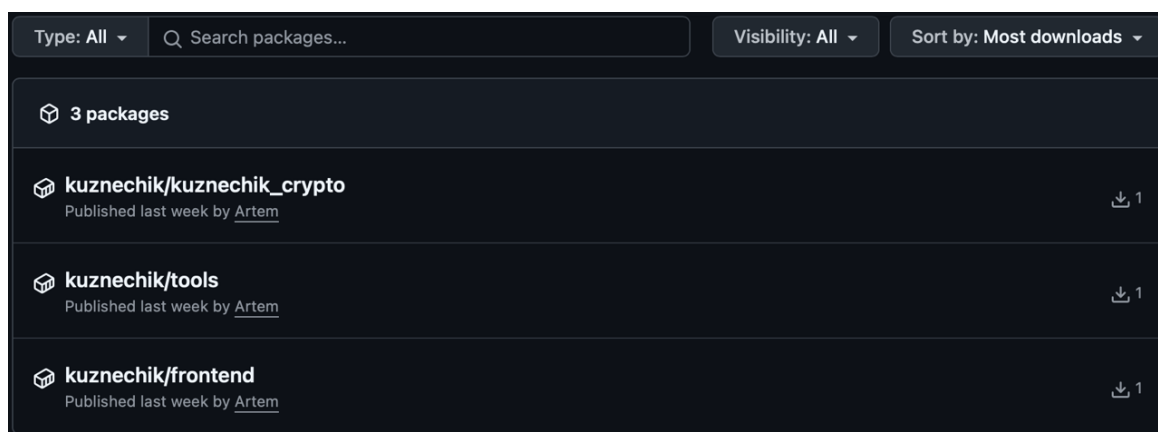


Рисунок 2.14 – Опубликованные Docker образы в GitHub Registry

2.9 Kubernetes манифесты

Для каждого микросервиса были разработаны Kubernetes-манифесты: Deployment и Service. В манифестах определены различные параметры, например стратегия развертывания, количество реплик, настройки проверок работоспособности и типы сервисов. Эти параметры напрямую влияют на стабильность работы компонентов и являются важной частью обеспечения отказоустойчивости микросервисной архитектуры.

Для микросервиса frontend выбрана стратегия развертывания RollingUpdate (листинг 2.4), что позволяет обновлять поды поэтапно, без остановки всего сервиса. Такой подход позволяет обновлять пользовательский интерфейс без недоступности.

Листинг 2.4 – Стратегия развертывание микросервиса frontend

```
strategy:
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 1
  type: RollingUpdate
```

Для бэкенд сервисов применяется стратегия Recreate (листинг 2.5). Данный подход исключает одновременное существование разных версий подов и предотвращает возможные несовместимости.

Листинг 2.5 – Стратегия развертывания микросервисов бэкенда

```
strategy:  
  type: Recreate
```

Каждому сервису выделены две реплики, что обеспечивает базовую горизонтальную отказоустойчивость: обе реплики работают параллельно, поддерживая стабильную работу сервиса. В случае сбоя одной из них, трафик переключается на оставшуюся, обеспечивая доступность для пользователя. При этом сбойный под автоматически восстанавливается. Как только он снова станет готов к работе, нагрузка постепенно перераспределится между обеими репликами.

Для всех микросервисов настроены проверки работоспособности livenessProbe, позволяющие Kubernetes автоматически выявлять и перезапускать неработающие поды. В микросервисе frontend проверка организована на основе доступности корневой страницы. Реализация livenessProbe показана в листинге 2.6.

Листинг 2.6 – Настройка livenessProbe для микросервиса frontend

```
livenessProbe:  
  httpGet:  
    path: /  
    port: 80  
  initialDelaySeconds: 5  
  periodSeconds: 5  
  timeoutSeconds: 1  
  successThreshold: 1  
  failureThreshold: 3
```

В двух бэкендах для этого выделена специальная конечная точка /health/liveness. В листинге 2.7 показана настройка livenessProbe для бэкендов.

Листинг 2.7 – Настройка livenessProbe для микросервисов бэкенда

```
livenessProbe:
  httpGet:
    path: /health/liveness
    port: 8000
  initialDelaySeconds: 5
  periodSeconds: 5
  timeoutSeconds: 1
  successThreshold: 1
  failureThreshold: 3
```

Типы сервисов настроены следующим образом:

- Сервис frontend использует тип NodePort, что обеспечивает возможность внешнего обращения к приложению;
- Сервисы бэкенда имеют тип ClusterIP, что обеспечивает их доступность только внутри кластера и исключает внешний доступ.

Архитектура приложения показана на рисунке 2.15. Пользователи работают с frontend, который обменивается данными с бэкендом, к которому прямого доступа у пользователей нет. Такая изоляция внутренних сервисов повышает надежность системы, снижая риск сбоев и обеспечивая защиту от несанкционированного доступа.

Архитектура приложения

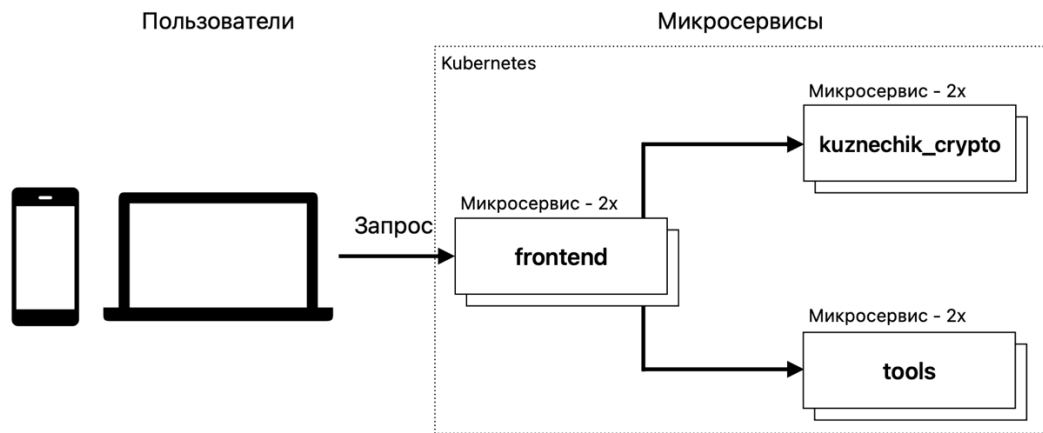


Рисунок 2.15 – Архитектура приложения

Поды, развернутые на основе Kubernetes манифестов показаны на рисунке 2.16.

Pods								
Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created ↑
frontend-deployment-5584df48bc-dw78b	ghcr.io/rezabungel/kuznechik/frontend:0.0.1	app: frontend component: web pod-template-hash: 5584df48bc	minikube	Running	0	1.00m	7.05Mi	17 minutes ago
frontend-deployment-5584df48bc-1ccqf	ghcr.io/rezabungel/kuznechik/frontend:0.0.1	app: frontend component: web pod-template-hash: 5584df48bc	minikube	Running	0	1.00m	7.09Mi	17 minutes ago
kuznechik-crypto-deployment-7c49675d45-bzp2l	ghcr.io/rezabungel/kuznechik/kuznechik_crypto:0.0.1	app: kuznechik-crypto component: api pod-template-hash: 7c49675d45	minikube	Running	0	12.00m	65.05Mi	17 minutes ago
kuznechik-crypto-deployment-7c49675d45-mpqkd	ghcr.io/rezabungel/kuznechik/kuznechik_crypto:0.0.1	app: kuznechik-crypto component: api pod-template-hash: 7c49675d45	minikube	Running	0	12.00m	65.13Mi	17 minutes ago
tools-deployment-9847bccb6-869kq	ghcr.io/rezabungel/kuznechik/tools:0.0.1	app: tools component: api pod-template-hash: 9847bccb6	minikube	Running	0	12.00m	64.49Mi	17 minutes ago
tools-deployment-9847bccb6-jm195	ghcr.io/rezabungel/kuznechik/tools:0.0.1	app: tools component: api pod-template-hash: 9847bccb6	minikube	Running	0	12.00m	64.51Mi	17 minutes ago

Рисунок 2.16 – Развернутые поды в Kubernetes

Использование различных стратегий развертывания, нескольких реплик, проверок работоспособности и разграничение типов сервисов обеспечивает отказоустойчивость, снижает время простоя и повышает безопасность микросервисной архитектуры.

2.10 Вывод по 2 главе

В данной главе научно-исследовательской работы была разработана отказоустойчивая информационная система – приложение с блочным шифрованием ГОСТ Р 34.12–2015 («Кузнечик»), основанное на микросервисной архитектуре.

Здесь представлено описание архитектуры приложения, приведено описание API веб-серверов бэкендов, продемонстрированы особенности реализации Docker образов микросервисов, а также рассмотрены основные элементы Kubernetes манифестов. Кроме того, были применены лучшие практики при построении CI-конвейера, автоматизирующего процессы сборки и публикации Docker образов.

ГЛАВА 3. ТЕСТИРОВАНИЕ РАЗРАБОТАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 Проверка работоспособности

В Kubernetes манифестах определены livenessProbe. Информацию о данной проверке можно найти в логах пода. На рисунке 3.1 показаны логи одного из подов микросервиса «tools».

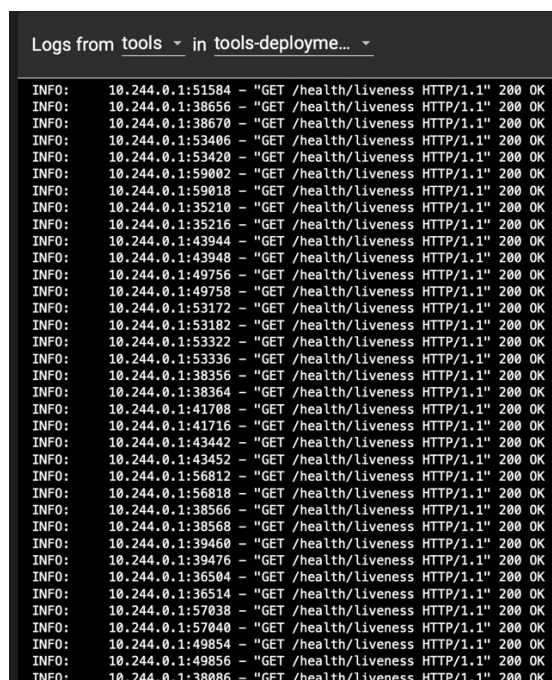


Рисунок 3.1 – Логи одного из подов микросервиса «tools»

Смоделировав ошибку, при которой приложение не может дальше работать, оно, соответственно, не сможет отвечать на проверки. Как показано на рисунке 3.2 проверки отработали неуспешно.

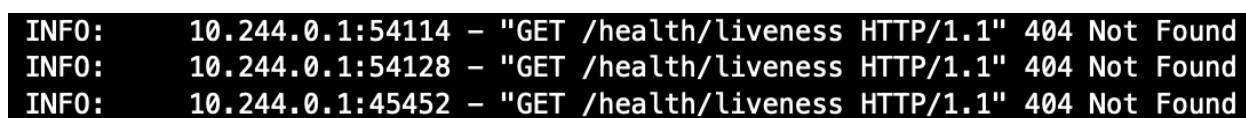


Рисунок 3.2 – Неуспешно отработанные проверки работоспособности

Неуспешно отработанные проверки работоспособности сигнализируют Kubernetes, что с приложением что-то не так, оно больше не отвечает. Kubernetes автоматически восстанавливает работу микросервиса, перезапустив его. Информация об этом показана на рисунке 3.3.

NAME	READY	STATUS	RESTARTS	AGE
frontend-deployment-5584df48bc-dw78b	1/1	Running	0	7h1m
frontend-deployment-5584df48bc-rccqf	1/1	Running	0	7h1m
kuznechik-crypto-deployment-7c49675d45-bzp2l	1/1	Running	0	7h1m
kuznechik-crypto-deployment-7c49675d45-mpqkd	1/1	Running	0	7h1m
tools-deployment-9847bccb6-jml95	1/1	Running	0	7h1m
tools-deployment-9847bccb6-r5wth	1/1	Running	0	68s
tools-deployment-9847bccb6-r5wth	1/1	Running	1 (3s ago)	94s

Рисунок 3.3 – Автоматический перезапуск пода с микросервисом

Так как запущено два экземпляра микросервиса «tools», конечный пользователь не заметит проблемы выхода из строя одного из них. Kubernetes попытается восстановить состояние системы, тем самым достигается отказоустойчивость системы, при возникновении каких-либо ошибок внутри приложения.

3.2 Недоступность микросервиса

Бывают ситуации, когда приложение не может продолжить свою работу по каким-либо причинам. Проверки работоспособности здесь не помогут, потому что приложение неработоспособно, Kubernetes лишь без конца будет перезапускать его.

Смоделируем ситуацию, когда микросервис «kuznechik_crypto» недоступен. Пользователь все еще может продолжить взаимодействовать с системой, но для него доступен только микросервис «tools». Данная ситуация показана на рисунке 3.4.

Рисунок 3.4 – Доступен только микросервис «tools»

В системе, реализованной как монолит, такой возможности нет, и система была бы полностью недоступна. Потеря части функционала информационной системы гораздо предпочтительнее, чем потеря всей системы. Микросервисная архитектура именно за счет разделения приложения на независимые компоненты позволяет минимизировать влияние отказа одного сервиса на работу всей системы, обеспечивая таким образом более высокую отказоустойчивость и доступность.

3.3 Масштабирование микросервисов

Не все сервисы находятся под одинаковой нагрузкой: одни востребованы больше, другие – меньше. Микросервисная архитектура позволяет увеличивать количество реплик конкретного сервиса. Это позволяет эффективно распределять ресурсы, горизонтально масштабируя только нужные в данный момент сервисы.

Если, например, большое количество пользователей начнет обращаться к микросервису «kuznechik_crypto», может возникнуть ситуация, что ответы будут возвращаться с высокой задержкой или сервис вовсе перестанет отвечать на запросы. Это негативно влияет на пользовательский опыт взаимодействия с системой. Для предотвращения подобных проблем можно временно увеличить количество реплик этого сервиса, как показано на рисунке 3.5.

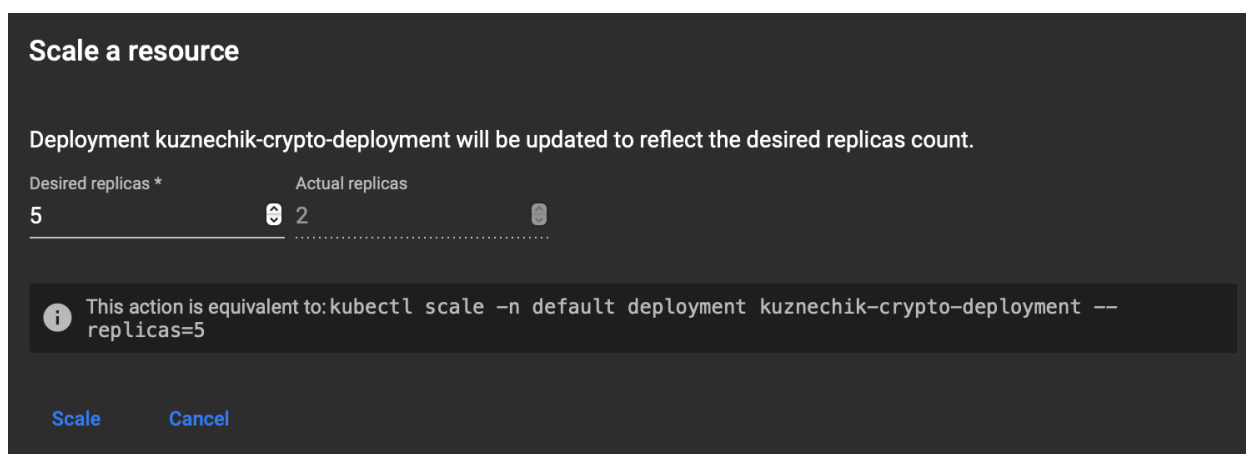


Рисунок 3.5 – Увеличение количества реплик микросервиса «kuznechik_crypto»

Горизонтальное масштабирование поддерживается всеми микросервисами системы, что повышает ее отказоустойчивость, особенно в периоды пиковых нагрузок.

3.4 Обновление системы

Система обладает возможностью обновления микросервиса «frontend» без недоступности. Это достигается путем использования стратегии RollingUpdate. Процесс обновления данного микросервиса показан на рисунке 3.6.

NAME	READY	STATUS	RESTARTS	AGE
frontend-deployment-5584df48bc-zkhwp	1/1	Running	0	9m49s
frontend-deployment-5584df48bc-zrjmg	1/1	Running	0	10m
kuznechik-crypto-deployment-7c49675d45-5nfms	1/1	Running	0	6m25s
kuznechik-crypto-deployment-7c49675d45-pbsfk	1/1	Running	0	6m25s
tools-deployment-9847bccb6-gbk8h	1/1	Running	0	8m18s
tools-deployment-9847bccb6-wnzms	1/1	Running	0	8m18s
frontend-deployment-77689c6775-4ptvw	0/1	Pending	0	0s
frontend-deployment-5584df48bc-zkhwp	1/1	Terminating	0	13m
frontend-deployment-77689c6775-4ptvw	0/1	Pending	0	0s
frontend-deployment-77689c6775-4ptvw	0/1	ContainerCreating	0	0s
frontend-deployment-77689c6775-wxqh4	0/1	Pending	0	0s
frontend-deployment-77689c6775-wxqh4	0/1	Pending	0	0s
frontend-deployment-77689c6775-wxqh4	0/1	ContainerCreating	0	0s
frontend-deployment-5584df48bc-zkhwp	0/1	Completed	0	13m
frontend-deployment-5584df48bc-zkhwp	0/1	Completed	0	13m
frontend-deployment-5584df48bc-zkhwp	0/1	Completed	0	13m
frontend-deployment-77689c6775-4ptvw	1/1	Running	0	3s
frontend-deployment-77689c6775-wxqh4	1/1	Running	0	5s
frontend-deployment-5584df48bc-zrjmg	1/1	Terminating	0	14m
frontend-deployment-5584df48bc-zrjmg	0/1	Completed	0	14m
frontend-deployment-5584df48bc-zrjmg	0/1	Completed	0	14m
frontend-deployment-5584df48bc-zrjmg	0/1	Completed	0	14m

Рисунок 3.6 – Обновление микросервиса «frontend» без недоступности

Микросервисы бэкенда обновляются по стратегии Recreate, при которой старые поды сначала удаляются, а затем создаются новые. Данный процесс для микросервиса «kuznechik_crypto» показан на рисунке 3.7. Такой подход не позволяет провести обновление без недоступности, но он исключает одновременное существование разных версий подов и предотвращает возможные несовместимости.

NAME	READY	STATUS	RESTARTS	AGE
frontend-deployment-77689c6775-4ptvw	1/1	Running	0	9m16s
frontend-deployment-77689c6775-wxqh4	1/1	Running	0	9m16s
kuznechik-crypto-deployment-7c49675d45-5nfms	1/1	Running	0	19m
kuznechik-crypto-deployment-7c49675d45-pbsfk	1/1	Running	0	19m
tools-deployment-9847bccb6-gbk8h	1/1	Running	0	21m
tools-deployment-9847bccb6-wnzms	1/1	Running	0	21m
kuznechik-crypto-deployment-7c49675d45-pbsfk	1/1	Terminating	0	20m
kuznechik-crypto-deployment-7c49675d45-5nfms	1/1	Terminating	0	20m
kuznechik-crypto-deployment-7c49675d45-pbsfk	0/1	Completed	0	20m
kuznechik-crypto-deployment-7c49675d45-5nfms	0/1	Completed	0	20m
kuznechik-crypto-deployment-5cff4b89c4-bvstz	0/1	Pending	0	0s
kuznechik-crypto-deployment-5cff4b89c4-6dfdx	0/1	Pending	0	0s
kuznechik-crypto-deployment-5cff4b89c4-bvstz	0/1	Pending	0	0s
kuznechik-crypto-deployment-5cff4b89c4-6dfdx	0/1	Pending	0	0s
kuznechik-crypto-deployment-5cff4b89c4-bvstz	0/1	ContainerCreating	0	0s
kuznechik-crypto-deployment-5cff4b89c4-6dfdx	0/1	ContainerCreating	0	0s
kuznechik-crypto-deployment-7c49675d45-5nfms	0/1	Completed	0	20m
kuznechik-crypto-deployment-7c49675d45-5nfms	0/1	Completed	0	20m
kuznechik-crypto-deployment-7c49675d45-pbsfk	0/1	Completed	0	20m
kuznechik-crypto-deployment-7c49675d45-pbsfk	0/1	Completed	0	20m
kuznechik-crypto-deployment-5cff4b89c4-bvstz	1/1	Running	0	4s
kuznechik-crypto-deployment-5cff4b89c4-6dfdx	1/1	Running	0	6s

Рисунок 3.7 – Обновление микросервиса «kuznechik_crypto» по стратегии Recreate

Выбор стратегии RollingUpdate для обновления микросервиса «frontend» обеспечивает плавную замену его экземпляров без остановки работы, что обеспечивает доступность пользователя к UI части сервиса. Это снижает риски простоев и улучшает пользовательский опыт за счет непрерывной работы интерфейса. В то же время, для микросервисов бэкенда применяется стратегия Recreate, которая гарантирует стабильность и согласованность данных, несмотря на временную недоступность. Такой подход повышает общую отказоустойчивость системы, сочетая непрерывность обслуживания и надежность.

3.5 Вывод по 3 главе

В данной главе научно-исследовательской работы была протестирована разработанная информационная система. Результаты тестирования показывают, что применение микросервисной архитектуры и использование инструментов Kubernetes позволяют эффективно обеспечивать отказоустойчивость системы. Автоматическое обнаружение сбоев с помощью проверок работоспособности (livenessProbe) и их автоматическое устранение

посредством перезапуска подов предотвращают длительные простои сервисов. Горизонтальное масштабирование микросервисов в периоды повышенной нагрузки позволяет поддерживать стабильную производительность и минимизировать задержки в ответах. Применение стратегии RollingUpdate для обновления микросервиса «frontend» обеспечивает непрерывную работу интерфейса без простоев, улучшая пользовательский опыт. В то же время стратегия Recreate, используемая для бэкенд сервисов, гарантирует согласованность данных, что повышает надежность системы.

Таким образом, в ходе тестирования продемонстрирована высокая эффективность современных архитектурных решений обеспечения отказоустойчивости на базе микросервисов и оркестрации контейнеров.

ЗАКЛЮЧЕНИЕ

Актуальность темы обеспечения отказоустойчивости в современных информационных системах обусловлена растущими требованиями к стабильности, доступности и непрерывности цифровых сервисов. Сбои в работе компонентов ИТ-инфраструктуры могут приводить к серьезным последствиям: потерям данных, снижению доверия пользователей и финансовым убыткам. В условиях постоянно увеличивающейся нагрузки и усложняющейся архитектуры приложений особенно важным становится использование современных подходов к построению отказоустойчивых систем.

Целью научно-исследовательской работы была разработка отказоустойчивой информационной системы, основанной на микросервисной архитектуре, с применением современных архитектурных решений и лучших практик обеспечения отказоустойчивости.

В ходе выполнения научно-исследовательской работы были получены следующие результаты.

1. Проведен анализ принципов и методов построения отказоустойчивых информационных систем, который показал, что эффективное обеспечение отказоустойчивости требует комплексного подхода, включающего как инфраструктурные, так и архитектурные решения.

2. Выполнен анализ лучших практик построения отказоустойчивых информационных систем, в ходе которого рассмотрены DevOps подходы, современные стратегии развертывания приложений, а также роль мониторинга и восстановления после сбоев в обеспечении устойчивой и надежной работы систем.

3. Разработана отказоустойчивая информационная система, которая представляется собой приложение на микросервисной архитектуре с применением современных архитектурных решений обеспечения отказоустойчивости на базе микросервисов и оркестрации контейнеров средствами Kubernetes. Система включает микросервисы пользовательского

интерфейса, бэкендов и CI-конвейер, обеспечивает горизонтальное масштабирование, автоматическое восстановление после сбоев и безопасное обновление компонентов.

4. Проведено тестирование разработанной информационной системы, показавшее ее устойчивость к отказам отдельных компонентов, способность к автоматическому восстановлению, поддержку горизонтального масштабирования микросервисов, а также непрерывную доступность пользовательского интерфейса при обновлении. Эти результаты подтверждают целесообразность использования микросервисного подхода, Docker контейнеризации и оркестрации Kubernetes для построения отказоустойчивых информационных систем.

Результатом проделанной работы является микросервисное приложение с блочным шифрованием ГОСТ Р 34.12–2015 («Кузнечик») на C++ и взаимодействием по REST API на Python.

Дальнейшее развитие системы предполагается за счет внедрения мониторинга и алертинга на базе Prometheus и Grafana, что позволит отслеживать ключевые метрики производительности, доступности и ошибок в работе микросервисов. Настраиваемые дашборды обеспечат визуализацию состояния системы, а алерты – своевременное информирование о сбоях. Это повысит устойчивость системы, упростит диагностику инцидентов и ускорит реагирование на них.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Что такое информационная безопасность и какие данные она охраняет [электронный ресурс] – URL: <https://cloud.vk.com/blog/informacionnaya-bezopasnost-i-kakie-dannye-ona-ohranyaet/> (дата обращения 26.04.2025).
2. Отказоустойчивая IT-инфраструктура: принципы и способы организации [электронный ресурс] – URL: <https://timeweb.cloud/tutorials/servers/otkazoustojchivaya-it-infrastruktura> (дата обращения 26.04.2025).
3. Соглашение об уровне обслуживания (SLA) [электронный ресурс] – URL: <https://yandex.cloud/ru/docs/glossary/sla> (дата обращения 26.04.2025).
4. Как повысить отказоустойчивость ИТ-инфраструктуры [электронный ресурс] – URL: <https://onlanta.ru/press/blog/kak-povysit-otkazoustoychivost-it-infrastruktury/> (дата обращения 26.04.2025).
5. Отказоустойчивые системы: зачем нужны и как построить [электронный ресурс] – URL: <https://habr.com/ru/companies/x-com/articles/697796/> (дата обращения 26.04.2025).
6. RAID [электронный ресурс] – URL: <https://ddos-guard.ru/technologies/raid> (дата обращения 28.04.2025).
7. Описание уровней RAID [электронный ресурс] – URL: <https://docs.itvgroup.ru/confluence/pages/viewpage.action?pageId=224299196> (дата обращения 28.04.2025).
8. RAID [электронный ресурс] – URL: <https://ru.wikipedia.org/wiki/RAID> (дата обращения 28.04.2025).
9. ПЛАН РЕЗЕРВНОГО КОПИРОВАНИЯ И АВАРИЙНОГО ВОССТАНОВЛЕНИЯ ДАННЫХ [электронный ресурс] – URL: <https://backupsolution.ru/backup-plan/> (дата обращения 29.04.2025).
10. Бэкап: что такое резервное копирование данных [электронный ресурс] – URL: <https://yandex.cloud/ru/docs/glossary/backup> (дата обращения 29.04.2025).

11. Uptime Institute Releases 2021 Global Data Center Survey [электронный ресурс] – URL: <https://facilityexecutive.com/uptime-institute-releases-2021-global-data-center-survey/> (дата обращения 29.04.2025).

12. Облачные решения для обеспечения отказоустойчивости: как гарантировать непрерывность бизнеса [электронный ресурс] – URL: <https://securitymedia.org/info/oblachnye-resheniya-dlya-obespecheniya-otkazoustoychivosti-kak-garantirovat-nepreryvnost-biznesa.html?ysclid=ma3syaeuhm353472770> (дата обращения 30.04.2025).

13. Отказоустойчивый кластер: что это такое и как работает [электронный ресурс] – URL: <https://www.ispsystem.ru/news/high-availability-cluster> (дата обращения 30.04.2025).

14. Монолитная и микросервисная архитектура. Сравнение [электронный ресурс] – URL: <https://habr.com/ru/companies/haulmont/articles/758780/> (дата обращения 01.05.2025).

15. Чем Docker отличается от виртуальной машины? [электронный ресурс] – URL: <https://wiki.merionet.ru/articles/chem-docker-on-otlichaetsya-ot-virtualnoj-mashiny> (дата обращения 01.05.2025).

16. K8s для начинающих [электронный ресурс] – URL: <https://www.reg.ru/blog/k8s-dlya-nachinayushchih> (дата обращения 01.05.2025).

17. Ускорьте свою базу данных: 7 проверенных методов масштабирования и оптимизации [электронный ресурс] – URL: <https://proglib.io/p/uskorte-svoyu-bazu-dannyh-7-proverennyh-metodov-masshtabirovaniya-i-optimizacii-2024-08-13> (дата обращения 02.05.2025).

18. Что такое DevOps: практики, методология, инструменты и почему бизнесу стоит его внедрять в разработку? [электронный ресурс] – URL: <https://timeweb.cloud/blog/devops-praktiki-metodologiya-instrumenty> (дата обращения 03.05.2025).

19. Стратегии развертывания в DevOps [электронный ресурс] – URL: <https://mws.ru/blog/strategii-razvertyvaniya-v-devops/> (дата обращения 03.05.2025).

20. ГОСТ Р 34.12—2015. Информационная технология. Криптографическая защита информации. Блочные шифры : национальный стандарт Российской Федерации : издание официальное : утвержден и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 19 июня 2015 г. № 749-ст : введен впервые : дата введения 2016-01-01 / разработан Центром защиты информации и специальной связи ФСБ России с участием Открытого акционерного общества «Информационные технологии и коммуникационные системы» (ОАО «ИнфоТеКС»). — Москва : Стандартинформ, 2015. — IV, 21с.